

Practicum introduction to Computational Mathematics

WEEK 4

Subject:

Computational Mathematics

Prepared by:

Aisyah Currents

25091397108

Class International/Number 2



INFORMATICS MANAGEMENT STUDY PROGRAM

FACULTY OF VOCATION

UNIVERSITAS NEGERI SURABAYA

YEAR 2025

Table of Contents

PRACTICUM Part 1: Installation Process of the Python Development Environment.....	3
PRACTICUM Part 2: An introduction to the theory of computational mathematics.....	3
PRACTICUM Part 3: Practical Work about Introduction to the Theory of Computational Mathematics.....	10
PRACTICUM Part 4: Application of Propositional Logic Laws.....	12
PRACTICUM Part 5: Application of the Theory of Variants of Conditional Propositions.....	15
PRACTICUM Part 6: Application of the Theory of Logical Inference and Validity.....	17
PRACTICUM Part 7: Practical Work about Propositional Logic and Variants of Implication.	20
PRACTICUM Part 8: Application of the Theory of Biconditional (Bi-implication), Inference, Arguments, Axioms, Theorems, Lemmas, and Corollaries.....	23
PRACTICUM Part 9: Case Study in the Theory of Biconditional (Bi-implication), Inference, Arguments, Axioms, Theorems, Lemmas, and Corollaries.....	27
PRACTICUM Part 10: Case Study Applying Operators in the Theory of Biconditional (Bi-implication), Inference, Arguments, Axioms, Theorems, Lemmas, and Corollaries.....	31
PRACTICUM Part 11: Practical Work about Biconditional (Bi-implication), Inference, Arguments, Axioms, Theorems, Lemmas, and Corollaries.....	34
PRACTICUM Part 12: Application of the Concepts of Definition of Sets, Representation of Sets, Cardinality, Empty Set, Subsets, and Equal Sets....	47
PRACTICUM Part 13: Application in Python of Definition of Sets, Representation of Sets, Cardinality, Empty Set, Subsets, and Equal Sets....	50
PRACTICUM Part 14: Case Study in the Theory of Definition of Sets, Representation of Sets, Cardinality, Empty Set, Subsets, and Equal Sets....	54
PRACTICUM Part 15: Practical Work about Definition of Sets, Representation of Sets, Cardinality, Empty Set, Subsets, and Equal Sets....	57

PRACTICUM Part 1: Installation Process of the Python Development Environment

The first step is to download the Python installer from the official website at <https://www.python.org>. After that, the installer file needs to be executed to start the installation process. During installation, it is important to configure the settings, such as choosing the installation path, adding Python to the system PATH, and adjusting optional components if necessary.



The screenshot shows a Windows terminal window with the following content:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  [Icon] powershell + v [Icon] [Icon] ... | [Icon] [Icon] X
PS C:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\Matematika Komputasi\week 04\Praktikum
Matkom> python --version
Python 3.13.3
PS C:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\Matematika Komputasi\week 04\Praktikum
Matkom>

```

Once the configuration is complete, continue with the installation until it is finished. To ensure that Python is installed correctly, open the terminal or command prompt and type the command `python --version`. As an additional step, a code editor or IDE such as VS Code, PyCharm, or Jupyter Notebook can be installed to provide a more comfortable programming environment.

PRACTICUM Part 2: An introduction to the theory of computational mathematics

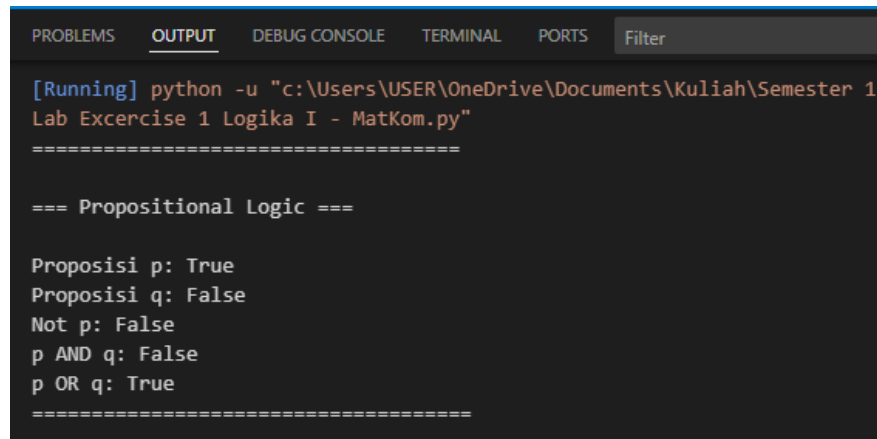
The Source Code for propositional logic

```
1. Lab Exercise 1 Logika I - MatKom.py X
1. Lab Exercise 1 Logika I - MatKom.py > propositional_logic
1 def propositional_logic():
2     print("=====\n")
3     print("=== Propositional Logic ===\n")
4     p = True # Contoh Proposisi p
5     q = False # Contoh Proposisi q
6     print(f"Proposisi p: {p}") # p adalah True
7     print(f"Proposisi q: {q}") # q adalah False
8     print(f"Not p: {not p}") # Negasi dari p
9     print(f"p AND q: {p and q}") # Konjungsi
10    print(f"p OR q: {p or q}") # Disjungsi
11    print("=====\n")
12
```

This program is designed to demonstrate how propositional logic works using basic logical operators in Python. Inside the `propositional_logic()` function,

two propositions are defined: $p = \text{True}$ and $q = \text{False}$. The program then prints the values of each proposition, followed by several logical operations. These operations include negation with $\text{not } p$, conjunction with $p \text{ and } q$, and disjunction with $p \text{ or } q$. To display the results, the program makes use of f-strings so that the values of the logical operations can be directly combined with explanatory text.

The Output:



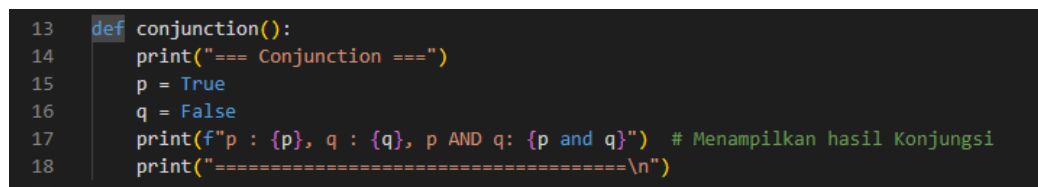
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter
[Running] python -u "c:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\
Lab Exercise 1 Logika I - MatKom.py"
=====

=== Propositional Logic ===

Proposisi p: True
Proposisi q: False
Not p: False
p AND q: False
p OR q: True
=====
```

After running, the program shows that the value of p is True and the value of q is False . When negation is applied to p , the result is False because the opposite of True is False . For the conjunction p and q , the result is False since conjunction is only True if both propositions are True . Meanwhile, for the disjunction p or q , the result is True because disjunction requires only one of the propositions to be True . In conclusion, the output demonstrates the basic rules of propositional logic: negation inverts a value, conjunction requires both propositions to be true, and disjunction is true if at least one proposition is true.

The Source Code for conjunction

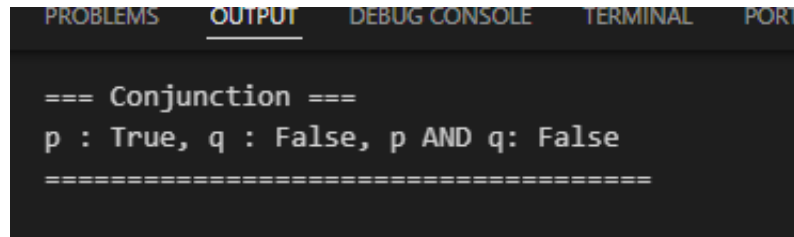


```
13 def conjunction():
14     print("=== Conjunction ===")
15     p = True
16     q = False
17     print(f"p : {p}, q : {q}, p AND q: {p and q}") # Menampilkan hasil Konjungsi
18     print("=====\n")
```

This function is defined to demonstrate the logical **AND** operator. Inside the function, two propositions are declared: $p = \text{True}$ and $q = \text{False}$. The program then prints their values alongside the result of the conjunction p and q . The **and** operator

in Python returns True only when both operands are True. Otherwise, it returns False. Finally, a separator line is printed to make the output clearer.

The Output

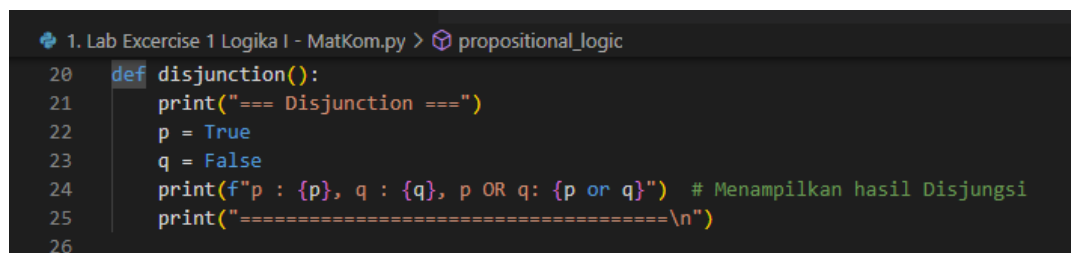


```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORT

=== Conjunction ===
p : True, q : False, p AND q: False
=====
```

The output shows that p is True and q is False. When evaluated with the AND operator, the result is False because conjunction requires both propositions to be True. Therefore, the output confirms the basic rule of conjunction: the result is only True if both p and q are True. Meanwhile in all other cases, it will be False.

The Source Code for disjunction

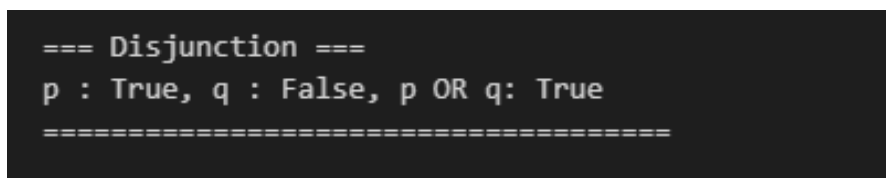


```
1. Lab Exercise 1 Logika I - MatKom.py > propositional_logic

20 def disjunction():
21     print("=== Disjunction ===")
22     p = True
23     q = False
24     print(f"p : {p}, q : {q}, p OR q: {p or q}") # Menampilkan hasil Disjungsi
25     print("=====\n")
26
```

This function is used to demonstrate the logical OR operator. Inside the function, two propositions are declared: p = True and q = False. The program then prints both propositions along with the result of the disjunction p or q. In Python, the or operator returns True if at least one of the operands is True. The function also includes separator lines to keep the output structured and easy to read.

The Output



```
=== Disjunction ===
p : True, q : False, p OR q: True
=====
```

Since p is True and q is False, the evaluation of p OR q results in True. This confirms the basic rule of disjunction: the outcome is only False if both propositions are False. Otherwise, if at least one proposition is True, the result will be True.

The Source Code for implication

```
26
27 def implication():
28     print("=== Implication ===")
29     p = True
30     q = False
31     print(f"p : {p}, q : {q}, p -> q: {not p or q}") # Menampilkan hasil Implikasi
32     print("=====\n")
33
```

This function is created to demonstrate the concept of logical implication. In propositional logic, an implication $p \rightarrow q$ can be expressed as $\neg p \vee q$ (not p or q). In this code, two propositions are defined, namely p = True and q = False. The program then calculates the result of the implication using the expression not p or q and displays it together with the values of p and q.

The Output

```
=== Implication ===
p : True, q : False, p -> q: False
=====
```

The output shows that when p is True and q is False, the implication $p \rightarrow q$ evaluates to False. This happens because not p becomes False, while q is also False, so the expression False or False results in False. This behavior is consistent with the fundamental rule of implication in logic, which states that an implication is only False if the first proposition is True while the second proposition is False. In all other cases, the implication is considered True.

The Source Code for biconditional

```
33
34 def biconditional():
35     print("=== Biconditional ===")
36     p = True
37     q = False
38     print(f"p : {p}, q : {q}, p <-> q: {(p and q) or (not p and not q)}")
39     print("=====\n")
40
```

This function is created to demonstrate the concept of biconditional logic. In propositional logic, a biconditional $p \leftrightarrow q$ means that both propositions must have the same truth value in order for the statement to be true. In Python, this can be expressed as $(p \text{ and } q) \text{ or } (\text{not } p \text{ and } \text{not } q)$. The first part $(p \text{ and } q)$ checks if both are true, while $(\text{not } p \text{ and } \text{not } q)$ checks if both are false. In this program, p is set to True and q is set to False. The calculation then evaluates $(p \text{ and } q) \text{ or } (\text{not } p \text{ and } \text{not } q)$. Since p and q is True and False, the result is False. The second part, not p and not q , becomes False and True, which is also False. Therefore, the overall result of the biconditional expression is False.

The Output

```
=== Biconditional ===  
p : True, q : False, p <-> q: False  
=====
```

The output confirms the logical rule of biconditional statements: the expression is true only if both propositions have the same truth value (both true or both false). If the truth values differ, as in this case ($p = \text{True}$, $q = \text{False}$), the biconditional evaluates to False.

The Source Code exclusive disjunction

```
41 def exclusive_disjunction():  
42     print("=== Exclusive Disjunction ===")  
43     p = True  
44     q = False  
45     # Disjungsi Eksklusif (XOR)  
46     xor_result = (p or q) and not (p and q) # XOR operator  
47     print(f"p : {p}, q : {q}, p XOR q: {xor_result}") # Menampilkan hasil XOR  
48     print("=====\n")
```

This function is designed to demonstrate the concept of exclusive disjunction (XOR) in propositional logic. In logic, XOR means that the result will be true if exactly one of the propositions is true, but false if both have the same truth value. In this code, the XOR operation is represented as $(p \text{ or } q) \text{ and } \text{not } (p \text{ and } q)$. The first part $(p \text{ or } q)$ ensures that at least one proposition is true, while $\text{not } (p \text{ and } q)$ ensures that both cannot be true at the same time. In this program, p is set to True and q is set to False. When evaluated, $(p \text{ or } q)$ becomes True since at least one of them is true. At the same time, $(p \text{ and } q)$ is False, so $\text{not } (p \text{ and } q)$ becomes True. Combining both results, the whole expression $(p \text{ or } q) \text{ and } \text{not } (p \text{ and } q)$ evaluates to True.

The Output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORT

=== Exclusive Disjunction ===
p : True, q : False, p XOR q: True
=====
```

The output confirms the rule of XOR: the result is true if one and only one of the propositions is true, and false if both are true or both are false. In this case, because p and q have different truth values (True and False), the XOR expression evaluates to True.

The Source Code for combine propositions

```
→ Praktikum Matkom

1. Lab Exercise 1 Logika I - MatKom.py X
1. Lab Exercise 1 Logika I - MatKom.py > propositional_logic

50 def combine_propositions():
51     print("=== Combine Propositions ===")
52     p = True
53     q = False
54     r = True
55     # Menggunakan operator logika
56     combined = (p and q) or (not r) # Kombinasi beberapa proposisi
57     print(f"p : {p}, q : {q}, r : {r}, (p AND q) OR (NOT r): {combined}")
58     print("=====\n")
```

This function is created to demonstrate how multiple logical propositions can be combined into one complex expression. In this function, three propositions are defined: p = True, q = False, and r = True. The logical expression used is (p and q) or (not r). Here, (p and q) represents the conjunction of p and q, while (not r) is the negation of r. These two parts are then combined using the or operator. When the program runs, the first part (p and q) is evaluated as True and False, which results in False. The second part (not r) becomes not True, which is also False. Finally, combining both with the or operator gives False or False, which results in False.

The Output

```
=== Combine Propositions ===
p : True, q : False, r : True, (p AND q) OR (NOT r): False
=====
```


The output confirms this evaluation by showing that with $p = \text{True}$, $q = \text{False}$, and $r = \text{True}$, the expression $(p \text{ AND } q) \text{ OR } (\text{NOT } r)$ evaluates to False. This illustrates how more complex logical statements can be built by combining simple propositions with different logical operators.

The Source Code for truth table

```

60 def truth_table():
61     print("=== Truth Table ===")
62     print("p\tq\tp AND q\t\tNot p\t\tp -> q\t\tp <-> q")
63     for p in [True, False]:
64         for q in [True, False]:
65             conjunction = p and q
66             disjunction = p or q
67             implication = not p or q
68             biconditional = (p and q) or (not p and not q)
69             print(f"{p}\t{q}\t{conjunction}\t\t{disjunction}\t\t{not p}\t\t{implication}\t\t{biconditional}")
70     print("=====\n")
71

```

This function is used to display a complete truth table for two propositions, p and q , along with several logical operations. The program evaluates conjunction ($p \text{ AND } q$), disjunction ($p \text{ OR } q$), negation ($\text{NOT } p$), implication ($p \rightarrow q$), and biconditional ($p \leftrightarrow q$) for every possible combination of truth values of p and q . From the output, we can see that when both p and q are true, all operations give true results. When p is true and q is false, only disjunction is true, while implication and biconditional become false. When p is false and q is true, disjunction and implication are true, but biconditional is false. Finally, when both p and q are false, implication and biconditional are true, while conjunction and disjunction are false. This table clearly shows the fundamental behavior of logical operators and how they depend on the truth values of the propositions involved.

The Output

```

=== Truth Table ===
p  q  p AND q  p OR q  Not p  p -> q  p <-> q
True True   True   True   True   False   True
True False  False  True   True   False   False
False True   False  True   True   True    True
False False  False  False  True   True    True
=====

```

The output of the `truth_table()` function shows how logical operators behave with all combinations of p and q . When both are true, all operations except negation are true. When p is true and q is false, conjunction, implication, and biconditional are false, while disjunction is true. When p is false and q is true, disjunction and implication are true, but conjunction and biconditional are false. Finally, when both are false, conjunction and disjunction are false, while implication and biconditional are true. This summarizes the core rules of propositional logic.

The Source Code for main execution

```

71
72 if __name__ == "__main__":
73
74     # Course_contract()
75     propositional_logic()
76
77     conjunction()
78     disjunction()
79     implication()
80     biconditional()
81
82     exclusive_disjunction()
83     combine_propositions()
84
85     truth_table()
86

```

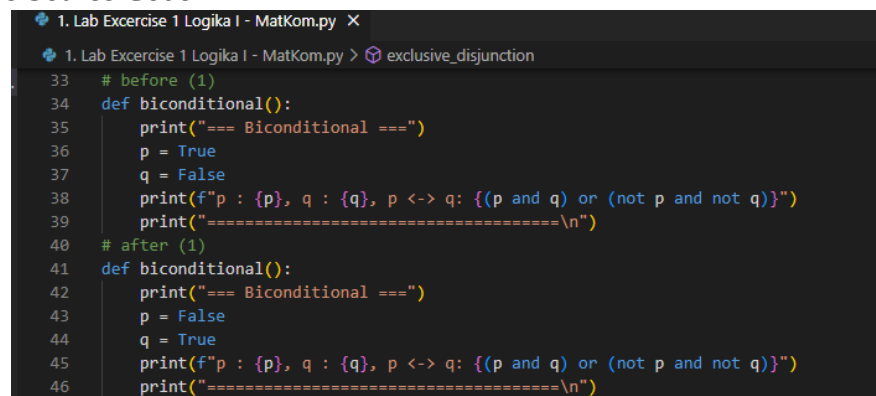
This block acts as the driver program that executes all the logic functions sequentially. When you run the file, it will display the outputs of each logical operator one after another, giving you both individual demonstrations and a complete truth table at the end.

PRACTICUM Part 3: Practical Work about Introduction to the Theory of Computational Mathematics

Modifying values in two applications of computational mathematics theory, including:

1. Biconditional Function

The Source Code



```

1. Lab Exercise 1 Logika I - MatKom.py X
1. Lab Exercise 1 Logika I - MatKom.py > exclusive_disjunction
33 # before (1)
34 def biconditional():
35     print("=== Biconditional ===")
36     p = True
37     q = False
38     print(f"p : {p}, q : {q}, p <-> q: {(p and q) or (not p and not q)}")
39     print("===== \n")
40 # after (1)
41 def biconditional():
42     print("=== Biconditional ===")
43     p = False
44     q = True
45     print(f"p : {p}, q : {q}, p <-> q: {(p and q) or (not p and not q)}")
46     print("===== \n")

```

The function `biconditional()` is designed to demonstrate the biconditional logical operation ($p \leftrightarrow q$). In this function, two Boolean variables are defined: `p` and `q`. The biconditional statement is true if both variables have the same truth value (both true or both false) and false otherwise. The function prints the values of `p` and `q` and then evaluates the biconditional expression using the formula $(p \text{ and } q) \text{ or } (\text{not } p \text{ and not } q)$.

The Output

```

=== Biconditional ===
p : False, q : True, p <-> q: False
=====

```

The program sets $p = \text{False}$ and $q = \text{True}$. When substituted into the biconditional formula $(p \text{ and } q) \text{ or } (\text{not } p \text{ and not } q)$, the result is False, because the values of p and q are not the same. Therefore, the output correctly shows:

The Analysis

This result illustrates the principle of biconditional logic, which only holds true when both propositions share identical truth values. In the given case, since p and q are different, the biconditional evaluates to false. This confirms the correctness of the logical relationship: a biconditional requires equivalence between two statements. If one is true and the other is false, then the overall expression must be false.

2. Exclusive Disjunction Function

The Source Code

```

1. Lab Exercise 1 Logika I - MatKom.py X
1. Lab Exercise 1 Logika I - MatKom.py > exclusive_disjunction
47 # before (2)
48 def exclusive_disjunction():
49     print("=== Exclusive Disjunction ===")
50     p = True
51     q = False
52     # Disjungsi Eksklusif (XOR)
53     xor_result = (p or q) and not (p and q) # XOR operator
54     print(f"p : {p}, q : {q}, p XOR q: {xor_result}") # Menampilkan hasil XOR
55     print("=====\n")
56 # after (2)
57 def exclusive_disjunction():
58     print("=== Exclusive Disjunction ===")
59     p = False
60     q = True
61     # Disjungsi Eksklusif (XOR)
62     xor_result = (p or q) and not (p and q) # XOR operator
63     print(f"p : {p}, q : {q}, p XOR q: {xor_result}") # Menampilkan hasil XOR
64     print("=====\n")
65

```

The function `exclusive_disjunction()` demonstrates the exclusive disjunction (XOR) logical operator. Two Boolean variables p and q are defined, and the XOR result is calculated using the expression $(p \text{ or } q) \text{ and not } (p \text{ and } q)$. This formula ensures that the result is true only if exactly one of the variables is true, but false if both have the same value (both true or both false). The program then prints the values of p and q alongside the calculated XOR result.

The Output

```

=== Exclusive Disjunction ===
p : False, q : True, p XOR q: True
=====

```

In this case, $p = \text{False}$ and $q = \text{True}$. Substituting into the formula $(p \text{ or } q)$ and not $(p \text{ and } q)$ results in True , because one of the inputs is true while the other is false.

The Analysis

This output confirms the definition of exclusive disjunction. XOR highlights situations where two propositions differ: the expression evaluates to true only when one input is true and the other is false. If both inputs match (both true or both false), the result is false. This makes XOR useful for representing conditions of inequality between two logical values.

PRACTICUM Part 4: Application of Propositional Logic Laws

The Source Code for identity law

```

2. Lab Exercise 2 Logika II - MatKom.py > ...
1  def identity_law():
2      print("=====\n")
3      print("=== Identity Law ===\n")
4      p = True
5      print(f"p OR False: {p or False}")
6      print(f"p AND True: {p and True}")
7      print("=====\n")
8

```

The function `identity_law()` illustrates the Identity Law in propositional logic. It sets $p = \text{True}$ and then evaluates two expressions: $p \text{ OR False}$ and $p \text{ AND True}$. According to the identity law, $p \text{ OR False}$ should always equal p , and $p \text{ AND True}$ should also equal p . The program prints these results to show the concept in action.

The Output

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  POR
[Running] python -u "c:\Users\USER\OneDrive\D
Lab Exercise 2 Logika II - MatKom.py"
=====

=== Identity Law ===

p OR False: True
p AND True: True
=====

```

With $p = \text{True}$, the expression $p \text{ OR } \text{False}$ produces True because $\text{True} \vee \text{False} = \text{True}$. Similarly, the expression $p \text{ AND } \text{True}$ results in True because $\text{True} \wedge \text{True} = \text{True}$. The output matches the expected results of the identity law.

The Source Code for commutative law

```
9 def commutative_law():
10     print("=== Commutative Law ===")
11     p = True
12     q = False
13     print(f"p OR q: {p or q}")
14     print(f"q OR p: {q or p}")
15     print(f"p AND q: {p and q}")
16     print(f"p AND q: {p and q}")
17     print("=====\n")
18
```

The function `commutative_law()` demonstrates the Commutative Law in logic. This law states that the order of operands in OR ($p \vee q = q \vee p$) and AND ($p \wedge q = q \wedge p$) does not affect the result. In the program, the variables are defined as $p = \text{True}$ and $q = \text{False}$. The code prints the results of $p \text{ OR } q$, $q \text{ OR } p$, $p \text{ AND } q$, and $q \text{ AND } p$ to show that the expressions remain equal regardless of the order.

The Output

```
=== Commutative Law ===
p OR q: True
q OR p: True
p AND q: False
p AND q: False
=====
```

With $p = \text{True}$ and $q = \text{False}$, both $p \text{ OR } q$ and $q \text{ OR } p$ evaluate to True . Meanwhile, both $p \text{ AND } q$ and $q \text{ AND } p$ evaluate to False . The output confirms that swapping the order of the operands does not change the result, consistent with the commutative law.

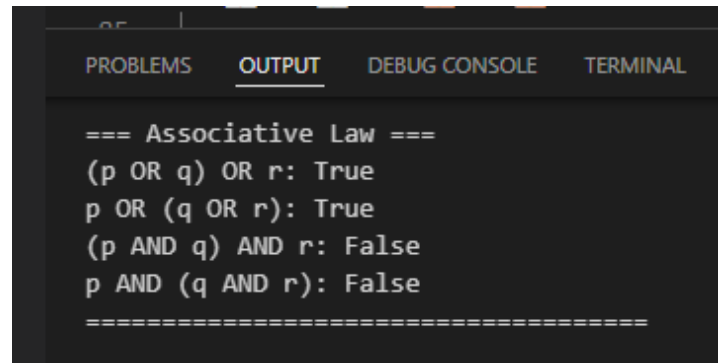
The Source Code for associative law

```
19 def associative_law():
20     print("=== Associative Law ===")
21     p = True
22     q = False
23     r = True
24     print(f"(p OR q) OR r: {(p or q) or r}")
25     print(f"p OR (q OR r): {p or (q or r)}")
26     print(f"(p AND q) AND r: {(p and q) and r}")
27     print(f"p AND (q AND r): {p and (q and r)}")
28     print("=====\n")
29
```

The function `associative_law()` demonstrates the Associative Law in propositional logic. This law states that when three propositions are combined with

OR $(p \vee (q \vee r) = (p \vee q) \vee r)$ or AND $(p \wedge (q \wedge r) = (p \wedge q) \wedge r)$, the grouping of parentheses does not affect the result. In the program, the variables are defined as $p = \text{True}$, $q = \text{False}$, and $r = \text{True}$. The function prints the results of both OR and AND operations with different groupings to show that the outcomes remain equal.

The Output

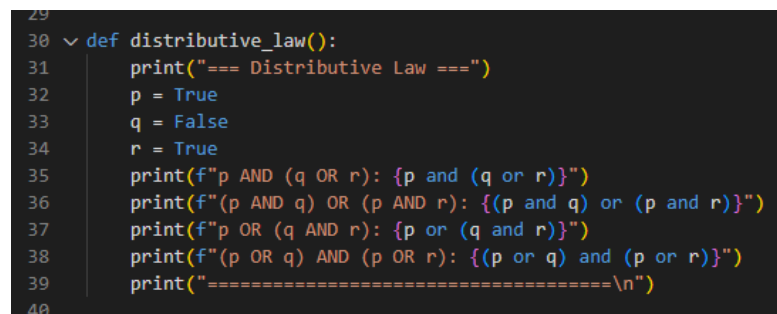


```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

=== Associative Law ===
(p OR q) OR r: True
p OR (q OR r): True
(p AND q) AND r: False
p AND (q AND r): False
=====
```

With $p = \text{True}$, $q = \text{False}$, and $r = \text{True}$, both expressions $(p \vee q) \vee r$ and $p \vee (q \vee r)$ evaluate to True . Similarly, $(p \wedge q) \wedge r$ and $p \wedge (q \wedge r)$ both evaluate to False . The output confirms that the grouping of the operations does not change the logical result, consistent with the associative law.

The Source Code for distributive law



```
29
30 def distributive_law():
31     print("=== Distributive Law ===")
32     p = True
33     q = False
34     r = True
35     print(f"p AND (q OR r): {p and (q or r)}")
36     print(f"(p AND q) OR (p AND r): {(p and q) or (p and r)}")
37     print(f"p OR (q AND r): {p or (q and r)}")
38     print(f"(p OR q) AND (p OR r): {(p or q) and (p or r)}")
39     print("=====\\n")
40
```

The function `distributive_law()` illustrates the Distributive Law in propositional logic. This law states that logical AND distributes over OR and vice versa. Specifically, $p \wedge (q \vee r) = (p \wedge q) \vee (p \wedge r)$ and $p \vee (q \wedge r) = (p \vee q) \wedge (p \vee r)$. In the program, the variables are defined as $p = \text{True}$, $q = \text{False}$, and $r = \text{True}$. The code prints the results of both distributive equivalences to confirm that the expressions on both sides are logically the same.

The Output

```

=== Distributive Law ===
p AND (q OR r): True
(p AND q) OR (p AND r): True
p OR (q AND r): True
(p OR q) AND (p OR r): True
=====

```

With $p = \text{True}$, $q = \text{False}$, and $r = \text{True}$, the expression $p \wedge (q \vee r)$ evaluates to True , which matches $(p \wedge q) \vee (p \wedge r)$ as True . Similarly, $p \vee (q \wedge r)$ evaluates to True , and $(p \vee q) \wedge (p \vee r)$ also results in True . The output verifies the distributive law in both cases, showing that the equivalences hold.

The Source Code for main execution

```

40
41 ✓ if __name__ == "__main__":
42
43     identity_law()
44     commutative_law()
45     associative_law()
46     distributive_law()

```

This block serves as the driver program that runs all the defined logic functions in sequence. When the file is executed, it calls each function (`identity_law()`, `commutative_law()`, `associative_law()`, and `distributive_law()`), displaying their outputs one after another. This structure not only provides individual demonstrations of each logical law but also presents a complete overview of how the different laws operate together in a single execution flow.

PRACTICUM Part 5: Application of the Theory of Variants of Conditional Propositions

The Source Code for converse

```

41 def converse():
42     print("=== Converse (Konversi) ===")
43     p = True
44     q = False
45     print(f"q -> p: {q} -> {p}")
46     print("=====\n")
47

```

The function `converse()` demonstrates the **Converse** of a conditional statement. If the original implication is $p \rightarrow q$, then its converse is $q \rightarrow p$. In the program, the variables are defined as $p = \text{True}$ and $q = \text{False}$. The function then

prints the result of the converse expression, showing how the truth values change when the direction of the implication is reversed.

The Output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

=== Converse (Konversi) ===
q -> p: False -> True
=====
```

With $p = \text{True}$ and $q = \text{False}$, the converse $q \rightarrow p$ is evaluated as True . This is because in logic, an implication with a false antecedent ($q = \text{False}$) is always true, regardless of the value of the consequent (p).

The Source Code for inverse

```
47
48 def inverse():
49     print("=== Inverse (Invers) ===")
50     p = True
51     q = False
52     print(f"~p -> ~q: {not p} -> {not q}")
53     print("=====\\n")
54
```

The function `inverse()` illustrates the Inverse of a conditional statement. If the original implication is $p \rightarrow q$, then its inverse is $\neg p \rightarrow \neg q$. In the program, the variables are defined as $p = \text{True}$ and $q = \text{False}$. The function then prints the result of the inverse expression ($\neg p \rightarrow \neg q$).

The Output

```
=== Inverse (Invers) ===
~p -> ~q: False -> True
=====
```

With $p = \text{True}$ and $q = \text{False}$, we get $\neg p = \text{False}$ and $\neg q = \text{True}$. Since an implication with a false antecedent ($\neg p$) is always true, the result is True .

The Source Code for contrapositive

```
54
55 def contrapositive():
56     print("=== Contrapositive (Kontrapositif) ===")
57     p = True
58     q = False
59     print(f"~q -> ~p: {not q} -> {not p}")
60     print("=====\\n")
61
```


The function `contrapositive()` demonstrates the Contrapositive of a conditional statement. For an implication $p \rightarrow q$, the contrapositive is $\neg q \rightarrow \neg p$. In the program, $p = \text{True}$ and $q = \text{False}$, and the code prints the result of evaluating $(\neg q \rightarrow \neg p)$.

The Output

```
=== Contrapositive (Kontrapositif) ===  
~q -> ~p: True -> False  
=====
```

With $p = \text{True}$ and $q = \text{False}$, we have $\neg q = \text{True}$ and $\neg p = \text{False}$. The implication $\neg q \rightarrow \neg p$ is therefore False, since a true antecedent leading to a false consequent makes the whole statement false.

The Source Code for main execution

```
62 if __name__ == "__main__":  
63     identity_law()  
64     commutative_law()  
65     associative_law()  
66     distributive_law()  
67       
68       
69     converse()  
70     inverse()  
71     contrapositive()
```

This block functions as the main driver program that executes all the logic-related functions in order. When the file is run, it calls the functions for identity, commutative, associative, and distributive laws, followed by converse, inverse, and contrapositive. The sequential execution ensures that the results of each logical concept are displayed one after another, giving a complete demonstration of propositional logic laws and their variations in a single run.

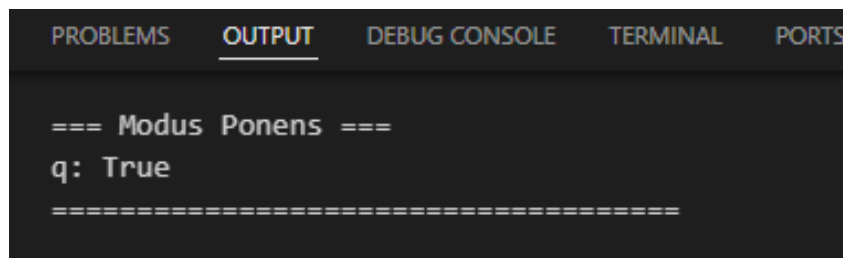
PRACTICUM Part 6: Application of the Theory of Logical Inference and Validity

The Source Code for modus ponens

```
62 def modus_ponens():  
63     print("=== Modus Ponens ===")  
64     p = True  
65     q = True  
66     print(f"q: {q}")  
67     print("=====\n")  
68
```

The function `modus_ponens()` demonstrates the logical inference rule known as Modus Ponens. This rule states that if $p \rightarrow q$ is true and p is true, then q must also be true. In the program, the variables are set as $p = \text{True}$ and $q = \text{True}$. The function then prints the value of q , showing the conclusion that follows from the premises.

The Output

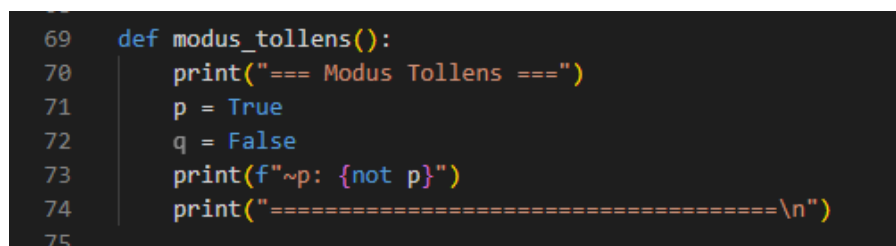


```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

=== Modus Ponens ===
q: True
=====
```

The output shows $q: \text{True}$, which is consistent with the rule of Modus Ponens. Since the premise p is true and the implication $p \rightarrow q$ is assumed to be valid, the conclusion must also be true. Therefore, the result confirms that q is true, demonstrating how Modus Ponens ensures a logically sound inference from the given premises.

The Source Code for modus Tollens



```
69 def modus_tollens():
70     print("=== Modus Tollens ===")
71     p = True
72     q = False
73     print(f"~p: {not p}")
74     print("=====\n")
75
```

The function `modus_tollens()` is intended to illustrate the Modus Tollens rule: if $p \rightarrow q$ is true and q is false, then p must be false ($\neg p$). In the code, the variables are set as $p = \text{True}$ and $q = \text{False}$, and the function prints $\neg p$ using `not p`. While the implication $p \rightarrow q$ isn't explicitly computed, the print focuses on the conclusion part of Modus Tollens (the negation of p) given the setup.

The Output



```
=== Modus Tollens ===
~p: False
=====
```

The output shows $\neg p: \text{False}$. Because p is assigned `True`, its negation $\neg p$ evaluates to `False`. This matches what the program prints: it directly reports the negated value of p under the Modus Tollens scenario where q is false.

The Source Code for syllogism

```
75
76 def syllogism():
77     print("=== Syllogism ===")
78     p = True
79     q = True
80     r = False
81     print(f"p -> r: {p} -> {r}")
82     print("=====\n")
83
```

The function `syllogism()` illustrates the concept of a Hypothetical Syllogism. In this type of logical reasoning, if $p \rightarrow q$ and $q \rightarrow r$ are true, then $p \rightarrow r$ must also be true. In the program, the variables are defined as $p = \text{True}$, $q = \text{True}$, and $r = \text{False}$. The function then prints the relationship between p and r , showing the result of the implication $p \rightarrow r$.

The Output

```
=== Syllogism ===
p -> r: True -> False
=====

[Done] exited with code=0 in 17.128 seconds
```

The output shows $p \rightarrow r: \text{True} \rightarrow \text{False}$. Because p is true and r is false, the implication $p \rightarrow r$ evaluates to False. This is correct according to propositional logic, since a true antecedent leading to a false consequent always makes the implication false.

The Source Code for main execution

```
84 if __name__ == "__main__":
85
86     identity_law()
87     commutative_law()
88     associative_law()
89     distributive_law()
90
91     converse()
92     inverse()
93     contrapositive()
94
95     modus_ponens()
96     modus_tollens()
97     syllogism()
```

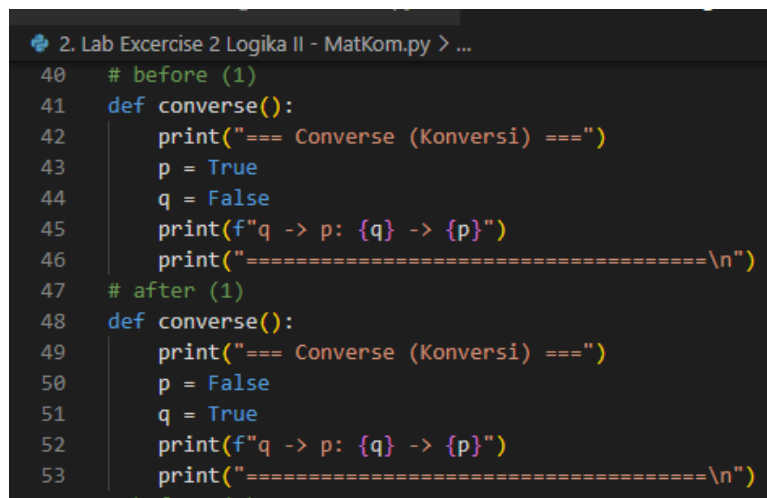
This block acts as the driver program that sequentially runs every function defined in the file. It executes inference rules like `modus_ponens()`, `modus_tollens()`, and `syllogism()`.

PRACTICUM Part 7: Practical Work about Propositional Logic and Variants of Implication.

Modifying values in Variants of Conditional Propositions, including:

1. Converse function

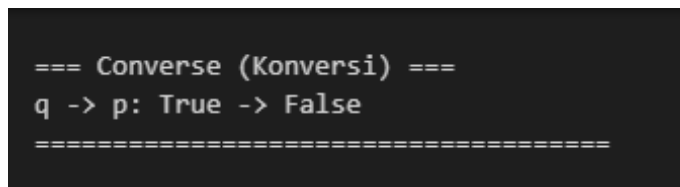
The Source Code



```
2. Lab Exercise 2 Logika II - MatKom.py > ...
40 # before (1)
41 def converse():
42     print("=== Converse (Konversi) ===")
43     p = True
44     q = False
45     print(f"q -> p: {q} -> {p}")
46     print("=====\n")
47 # after (1)
48 def converse():
49     print("=== Converse (Konversi) ===")
50     p = False
51     q = True
52     print(f"q -> p: {q} -> {p}")
53     print("=====\n")
```

The function is designed to demonstrate the converse of an implication. Normally, if the original statement is $p \rightarrow q$, the converse is written as $q \rightarrow p$. In the program, values are assigned to p and q in two different cases, and the code prints the result of evaluating the converse relationship between them.

The Output



```
=== Converse (Konversi) ===
q -> p: True -> False
=====
```

The output shows $q \rightarrow p: \text{True} \rightarrow \text{False}$, which happens when q is true and p is false. In propositional logic, an implication is false only when the antecedent is true while the consequent is false. Therefore, the program correctly displays that the converse evaluates to false in this situation.

The Analysis

This result illustrates that the truth value of a converse is not always the same as the original implication. In the original $p \rightarrow q$, if p were false, the implication might still be true. However, in the converse $q \rightarrow p$, when q is true but p is false, the statement fails. This shows how reversing the direction of implication can change the logical outcome and why converse must be analyzed separately from the original implication.

2. Inverse function

The Source Code

```
54 # before (2)
55 def inverse():
56     print("=== Inverse (Invers) ===")
57     p = True
58     q = False
59     print(f"~p -> ~q: {not p} -> {not q}")
60     print("=====\n")
61 # after (2)
62 def inverse():
63     print("=== Inverse (Invers) ===")
64     p = False
65     q = True
66     print(f"~p -> ~q: {not p} -> {not q}")
67     print("=====\n")
```

The function demonstrates the inverse of an implication. If the original statement is $p \rightarrow q$, then its inverse is $\neg p \rightarrow \neg q$. In the program, values of p and q are assigned in different test cases, and the code prints the result of evaluating the inverse expression. For the case shown, $p = \text{False}$ and $q = \text{True}$, so the program calculates $\neg p \rightarrow \neg q$.

The Output

```
=== Inverse (Invers) ===
~p -> ~q: True -> False
=====
```

The output displays $\neg p \rightarrow \neg q: \text{True} \rightarrow \text{False}$, which is correct because when $\neg p$ is true and $\neg q$ is false, the implication evaluates to false. This result confirms how the inverse of an implication can behave differently from the original statement and its converse.

The Analysis

The result of the inverse function shows that $\neg p \rightarrow \neg q$ can evaluate differently from the original implication $p \rightarrow q$. In this case, because p is

false and q is true, the values become $\neg p = \text{True}$ and $\neg q = \text{False}$. This makes the inverse implication false, since a true antecedent leading to a false consequent always invalidates the statement. The analysis highlights that the inverse does not necessarily have the same truth value as the original implication or its contrapositive, and therefore must be considered separately in logical reasoning.

3. Contrapositive

The Source Code

```
68 # before (3)
69 def contrapositive():
70     print("=== Contrapositive (Kontrapositif) ===")
71     p = True
72     q = False
73     print(f"~q -> ~p: {not q} -> {not p}")
74     print("=====\n")
75 # after (3)
76 def contrapositive():
77     print("=== Contrapositive (Kontrapositif) ===")
78     p = False
79     q = True
80     print(f"~q -> ~p: {not q} -> {not p}")
81     print("=====\n")
82
```

The function demonstrates the contrapositive of an implication. If the original statement is $p \rightarrow q$, its contrapositive is $\neg q \rightarrow \neg p$. In the code you set test values for p and q (shown in two cases) and then print the expression $\neg q \rightarrow \neg p$ by computing not q and not p. This showcases that we're not evaluating $p \rightarrow q$ directly, but its logically equivalent form, the contrapositive.

The Output

```
=== Contrapositive (Kontrapositif) ===
~q -> ~p: False -> True
=====
```

The printed line shows $\neg q \rightarrow \neg p$: False -> True, meaning $\neg q$ evaluates to False and $\neg p$ evaluates to True, so the implication $\neg q \rightarrow \neg p$ is True. In propositional logic, any implication with a false antecedent is true regardless of the consequent, which is why the program's result is true here. This output also reinforces the key fact that an implication and its contrapositive always share the same truth value.

The Analysis

The result of the contrapositive function shows that $\neg q \rightarrow \neg p$ preserves the same truth value as the original implication $p \rightarrow q$. In the program, $p = \text{False}$ and $q = \text{True}$, which gives $\neg q = \text{False}$ and $\neg p = \text{True}$. Since an implication with a false antecedent ($\neg q$) is always true regardless of the consequent, the expression evaluates to true. This confirms the logical principle that an implication and its contrapositive are always equivalent, meaning they will share the same truth value in every possible case.

PRACTICUM Part 8: Application of the Theory of Biconditional (Bi-implication), Inference, Arguments, Axioms, Theorems, Lemmas, and Corollaries

The Source Code for biconditional logic

```
3. Lab Exercise 3 Logika III - MatKom.py > ...
1  def biconditional_logic():
2      print("=== Biconditional (Bi-implication) ===")
3      p = True
4      q = False
5      # Bikonvensional (p <-> q) terjadi ketika p dan q memiliki nilai yang sama
6      biconditional = (p and q) or (not p and not q)
7      print(f"p: {p}, q: {q}, p <-> q: {biconditional}")
8      print("=====\n")
9
```

The function is designed to illustrate the biconditional (bi-implication) operator $p \leftrightarrow q$. In logic, this statement is true only if both p and q have the same truth value, either both true or both false. In the program, the variables are set as $p = \text{True}$ and $q = \text{False}$. The biconditional is then calculated using the formula $(p \text{ and } q) \text{ or } (\text{not } p \text{ and not } q)$, and the result is printed along with the values of p and q .

The Output

```
[Running] python -u "c:\Users\USER\OneDrive\Documen
Lab Exercise 3 Logika III - MatKom.py"
=== Biconditional (Bi-implication) ===
p: True, q: False, p <-> q: False
=====
```

The output shows $p: \text{True}$, $q: \text{False}$, $p \leftrightarrow q: \text{False}$. This is correct because when p and q differ in value (one is true, the other is false), the biconditional evaluates to false. The result demonstrates the fundamental rule of biconditional logic: it only holds true when both sides are equal in truth value.

The Source Code for error main execution

```

10 if __name__ == "__main__":
11     biconditional_logic()
12     inference()
13     argument()
14     axioms_theorems()

```

In the `if __name__ == "__main__":` block, the program simply calls those functions to print each analysis.

The Output

```

Traceback (most recent call last):
  File "c:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\Matematika Komputasi\week 04\Praktikum Matkom\3. Lab Exercise
  3 Logika III - MatKom.py", line 12, in <module>
    inference()
    ^^^^^^^^^
NameError: name 'inference' is not defined

[Done] exited with code=1 in 0.972 seconds

```

The screenshot shows `NameError: name 'inference' is not defined`, meaning Python can't find a function exactly named `inference` at runtime. This happens if the function was named differently (e.g., `inference_case` or `inference_all_cases`) or not imported. Fix by renaming the definition to `def inference(...):` or by changing the call in `main` to the correct existing name. After the names match, the script runs and prints the expected results for all four tasks.

The Source Code for inference

```

3. Lab Exercise 3 Logika III - MatKom.py > axioms_theorems
10 def inference():
11     print("=== Inference ===")
12     p = True
13     q = False
14     # Inferensi: Jika p -> q dan p benar, maka q juga harus benar
15     implication = not p or q
16     print(f"p: {p}, q: {q}, p -> q: {implication}")
17     if p and implication:
18         print(f"Dengan p benar, q harus: {q}")
19     print("=====\n")
20

```

The function sets `p=True` (sensor detects motion) and `q=False` (alarm silent), then computes the implication $p \rightarrow q$ as `implication = (not p) or q`. It prints the values of `p`, `q`, and the implication result. The `if p and implication:` line would print an extra note only when the rule holds while `p` is `True`; with `p=True`, `q=False` the condition is `False`, so that line doesn't run.

The Output


```

=== Inference ===
p: True, q: False, p -> q: False
=====

```

The output shows p: True, q: False, p -> q: False. This is correct because an implication is false only in the case p=True and q=False. Interpreted in context: the sensor detected motion but the alarm didn't ring, so the system violates the rule (gagal). If desired, add an else: print("Sistem gagal.") to state this explicitly.

The Source Code for argument

```

20
21 def argument():
22     print("=== Argument ===")
23     p = True
24     q = False
25     r = True
26     # Argumen dalam logika, contoh sederhana:
27     # Jika p -> q dan q -> r, maka p -> r
28     premise_1 = not p or q    # p -> q
29     premise_2 = not q or r    # q -> r
30     conclusion = not p or r   # p -> r
31
32     print(f"Premise 1 (p -> q): {premise_1}")
33     print(f"Premise 2 (q -> r): {premise_2}")
34     print(f"Conclusion (p -> r): {conclusion}")
35     print("=====\n")
36

```

The function sets p=True, q=False, r=True, then computes three implications: premise_1 = (not p) or q ($p \rightarrow q$), premise_2 = (not q) or r ($q \rightarrow r$), and conclusion = (not p) or r ($p \rightarrow r$). It only prints the truth values; it does not automatically judge the argument's validity.

The Output

```

=== Argument ===
Premise 1 (p -> q): False
Premise 2 (q -> r): True
Conclusion (p -> r): True
=====

```

For p=True, q=False, r=True: Premise 1 = False, Premise 2 = True, Conclusion = True. Because at least one premise is false, the inference pattern (hypothetical syllogism) is not valid in this case, the conclusion happens to be true, but it is not entailed by the premises.

The Source Code for axioms theoremas

```

3. Lab Exercise 3 Logika III - MatKom.py > axioms_theorems
37 def axioms_theorems():
38     print("=== Axioms, Theorems, Lemma, and Corollary ===")
39
40     # Axiom: Pernyataan dasar yang diasumsikan benar tanpa pembuktian
41     axiom = True # Contoh Aksioma yang selalu benar
42     print(f"Axiom: {axiom} (selalu dianggap benar)")
43
44     # Theorem: Pernyataan yang dibuktikan benar berdasarkan aksioma atau proposisi lain
45     theorem = axiom and True # Teorema dibuktikan dari aksioma
46     print(f"Theorem: {theorem} (berdasarkan aksioma)")
47
48     # Lemma: Pernyataan pembantu yang digunakan untuk membuktikan teorema
49     lemma = theorem # Lemma adalah bagian pembuktian teorema
50     print(f"Lemma: {lemma} (digunakan untuk membantu membuktikan teorema)")
51
52     # Corollary: Pernyataan yang mengikuti langsung dari teorema
53     corollary = theorem # Corollary adalah konsekuensi langsung dari teorema
54     print(f"Corollary: {corollary} (konsekuensi langsung dari teorema)")
55     print("=====\n")
56

```

The function sets four logical flags to illustrate roles in mathematics: axiom is assumed True (accepted without proof); theorem is derived from the axiom (theorem = axiom and True, so it follows when the axiom holds); lemma is a helper statement marked True (used to prove a theorem); and corollary is an immediate consequence of the theorem (corollary = theorem). The function then prints each label with its truth value.

The Output

```

=== Axioms, Theorems, Lemma, and Corollary ===
Axiom: True (selalu dianggap benar)
Theorem: True (berdasarkan aksioma)
Lemma: True (digunakan untuk membantu membuktikan teorema)
Corollary: True (konsekuensi langsung dari teorema)
=====

```

The output shows all four as True: the axiom is assumed true; the theorem evaluates to true because it depends on the true axiom; the lemma is set to true as a supporting fact; and the corollary is true because it directly follows from the true theorem.

The Source Code for the correct main execution

```

3. Lab Exercise 3 Logika III - MatKom.py > axioms_theorems
50
57 if __name__ == "__main__":
58     biconditional_logic()
59     inference()
60     argument()
61     axioms_theorems()

```

This block is the program's entry point. It runs only when the file is executed directly (not when imported). Inside, it calls the four routines in order, `biconditional_logic()`, `inference()`, `argument()`, and `axioms_theorems()`, so their analyses print sequentially. Make sure each function is defined above this block (or properly imported) and the names match exactly to avoid `NameError`.

PRACTICUM Part 9: Case Study in the Theory of Biconditional (Bi-implication), Inference, Arguments, Axioms, Theorems, Lemmas, and Corollaries

1. Case Study Biconditional (Bi-implication)

Suppose you have two network devices, *p* and *q*.

- *p*: Device A is functioning properly.
- *q*: Device B is functioning properly.

In biconditional logic, this statement means: "Device A functions if and only if Device B functions." This implies that both devices must either function at the same time or both fail at the same time.

If one device is working while the other is not, then the statement will be false.

The Source Code

```
4. Lab Exercise 4 Logika IV- MatKom.py > biconditional_logic_case
1 def biconditional_logic_case():
2     print("=== Studi Kasus Bikondisional (Bi-implikasi) ===")
3     perangkat_A = True # Perangkat A berfungsi
4     perangkat_B = False # Perangkat B tidak berfungsi
5
6     # A <-> B setara dengan (A & B) & (~A & ~B)
7     bikondisional = (perangkat_A and perangkat_B) or (not perangkat_A and not perangkat_B)
8
9     print(f"Perangkat A: {perangkat_A}, Perangkat B: {perangkat_B}, A <-> B: {bikondisional}")
10    print("=====\n")
11
```

The function sets `perangkat_A=True` and `perangkat_B=False`, then computes the biconditional with `bikondisional = (perangkat_A and perangkat_B) or (not perangkat_A and not perangkat_B)`, which is exactly $A \leftrightarrow B$. It prints both device states and the biconditional result.

The Output

```
[Running] python -u "c:\Users\USER\OneDrive\Documents\Kuliah\
Lab Exercise 4 Logika IV- MatKom.py"
=== Studi Kasus Bikondisional (Bi-implikasi) ===
Perangkat A: True, Perangkat B: False, A <-> B: False
=====
```

With A=True and B=False, the devices are in different states, so $A \leftrightarrow B$ evaluates to False (a biconditional is true only when both are True or both are False). Interpreted: the devices are not synchronized.

2. Case Study Inference

Suppose in a security system there is a statement: if the sensor detects motion (p), then the alarm will sound (q).

- **p:** The sensor detects motion.
- **q:** The alarm sounds.

If the sensor detects motion and we know that this sensor always works properly (the validity of $p \rightarrow q$), then it can be concluded that the alarm must sound.

The Source Code

```
12
13 def inference_case():
14     print("=== Studi Kasus Inferensi ===")
15     sensor_gerak = True      # Sensor mendeteksi gerakan
16     alarm_berbunyi = False  # Alarm belum berbunyi
17
18     # p -> q setara dengan ~p ∨ q
19     implication = (not sensor_gerak) or alarm_berbunyi
20
21     print(f"Sensor mendeteksi gerakan: {sensor_gerak}, Alarm berbunyi: {alarm_berbunyi}")
22     print(f"p -> q: {implication}")
23     if sensor_gerak and implication:
24         print("Alarm harus berbunyi!")
25     else:
26         print("Sistem gagal mendeteksi pergerakan!")
27     print("=====\n")
28
```

The function models the rule *if the sensor detects motion (p), the alarm must ring (q)*. It sets sensor_gerak=True and alarm_berbunyi=False, computes the implication $p \rightarrow q$ as $\text{implication} = (\text{not sensor_gerak}) \text{ or alarm_berbunyi}$, prints the values, then uses an if to report: if p is True and the implication holds, print "Alarm harus berbunyi!"; otherwise print "Sistem gagal mendeteksi pergerakan!".

The Output

```
=== Studi Kasus Inferensi ===
Sensor mendeteksi gerakan: True, Alarm berbunyi: False
p -> q: False
Sistem gagal mendeteksi pergerakan!
=====
```

The console shows $p \rightarrow q$: False because the only time an implication is false is when $p=\text{True}$ and $q=\text{False}$ —exactly the given inputs. Therefore the rule is violated and the else branch runs, printing “Sistem gagal mendeteksi pergerakan!”.

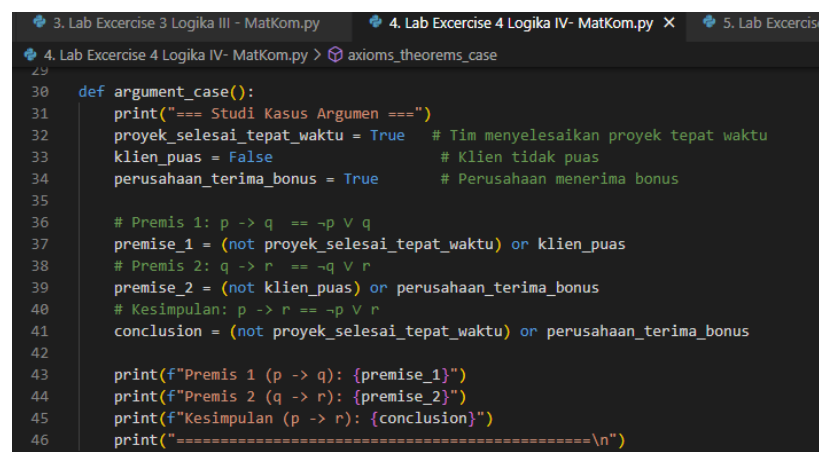
3. Case Study Argument

Suppose in a business process:

- If the team completes the project on time (p), then the client will be satisfied (q).
- If the client is satisfied (q), then the company will receive a bonus (r).

From these two premises, it can be concluded that if the project is completed on time, the company will receive a bonus.

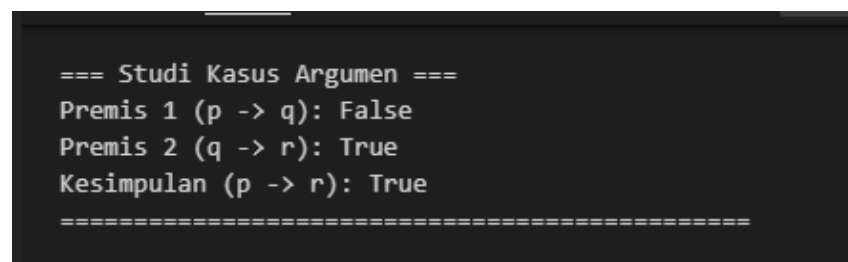
The Source Code



```
30 def argument_case():
31     print("=== Studi Kasus Argumen ===")
32     proyek_selesai_tepat_waktu = True # Tim menyelesaikan proyek tepat waktu
33     klien_puas = False # Klien tidak puas
34     perusahaan_terima_bonus = True # Perusahaan menerima bonus
35
36     # Premis 1:  $p \rightarrow q \equiv \neg p \vee q$ 
37     premise_1 = (not proyek_selesai_tepat_waktu) or klien_puas
38     # Premis 2:  $q \rightarrow r \equiv \neg q \vee r$ 
39     premise_2 = (not klien_puas) or perusahaan_terima_bonus
40     # Kesimpulan:  $p \rightarrow r \equiv \neg p \vee r$ 
41     conclusion = (not proyek_selesai_tepat_waktu) or perusahaan_terima_bonus
42
43     print(f"Premis 1 ( $p \rightarrow q$ ): {premise_1}")
44     print(f"Premis 2 ( $q \rightarrow r$ ): {premise_2}")
45     print(f"Kesimpulan ( $p \rightarrow r$ ): {conclusion}")
46     print("=====\n")
```

The function sets $p=\text{True}$ (project on time), $q=\text{False}$ (client not satisfied), and $r=\text{True}$ (company gets a bonus). It then computes three implications: $\text{premise_1} = (\text{not } p) \text{ or } q$ for $p \rightarrow q$, $\text{premise_2} = (\text{not } q) \text{ or } r$ for $q \rightarrow r$, and $\text{conclusion} = (\text{not } p) \text{ or } r$ for $p \rightarrow r$, and prints their truth values. This models the hypothetical-syllogism chain.

The Output



```
=== Studi Kasus Argumen ===
Premis 1 ( $p \rightarrow q$ ): False
Premis 2 ( $q \rightarrow r$ ): True
Kesimpulan ( $p \rightarrow r$ ): True
=====
```

With $p=\text{True}$, $q=\text{False}$, $r=\text{True}$, Premise 1 is False (an implication fails only when antecedent True and consequent False), Premise 2 is True (because not q is True), and the Conclusion is True (since $r=\text{True}$). Because a premise is false, the argument is not valid, the conclusion happens to be true but is not proven by the given premises.

4. Case Study Axioms, Theorems, Lemma, and Corollary

In a mathematical system, there are statements:

- Axiom: Every triangle has a total of 180 degrees (assumed to be true without proof).
- Theorem: If the triangle is a right triangle, then one of its angles is 90 degrees (proven from the axiom).
- Lemma: The angle opposite the longest side in a right triangle is 90 degrees (proven in order to help prove the theorem).
- Corollary: If an angle of a triangle is 90 degrees, then the triangle is a right triangle (a direct consequence of the theorem).

The Source Code

```
4. Lab Exercise 4 Logika IV- MatKom.py > axioms_theorems_case
48
49 def axioms_theorems_case():
50     print("=== Studi Kasus Aksioma, Teorema, Lemma, dan Corollary ===")
51     # Aksioma
52     aksioma = "Total sudut segitiga adalah 180 derajat"
53     print(f"Aksioma: {aksioma}")
54
55     # Teorema
56     teorema = "Segitiga siku-siku memiliki sudut 90 derajat"
57     print(f"Teorema: {teorema}")
58
59     # Lemma
60     lemma = "Sudut di seberang sisi terpanjang dalam segitiga siku-siku adalah 90 derajat"
61     print(f"Lemma: {lemma}")
62
63     # Corollary
64     corollary = "Jika segitiga memiliki sudut 90 derajat, maka segitiga tersebut adalah segitiga siku-siku"
65     print(f"Corollary: {corollary}")
66     print("=====\n")
67
```

The function defines four strings that describe core geometry facts: an axiom (sum of a triangle's angles is 180°), a theorem (a right triangle has a 90° angle), a lemma (in a right triangle, the angle opposite the longest side is 90°), and a corollary (if a triangle has a 90° angle, it is a right triangle). It simply uses f-strings to print each item; there is no calculation or branching.

The Output

```

=== Studi Kasus Aksioma, Teorema, Lemma, dan Corollary ===
Aksioma: Total sudut segitiga adalah 180 derajat
Teorema: Segitiga siku-siku memiliki sudut 90 derajat
Lemma: Sudut di seberang sisi terpanjang dalam segitiga siku-siku adalah 90 derajat
Corollary: Jika segitiga memiliki sudut 90 derajat, maka segitiga tersebut adalah segitiga siku-siku
=====

```

The console shows the four labeled statements exactly as defined, confirming the axiom, theorem, lemma, and corollary are displayed. The output is descriptive, meant to present the concepts, not to prove or compute them.

PRACTICUM Part 10: Case Study Applying Operators in the Theory of Biconditional (Bi-implication), Inference, Arguments, Axioms, Theorems, Lemmas, and Corollaries

1. Case Study Biconditional (Bi-implication)

In a company, an employee is considered eligible for a bonus if and only if they work more than 8 hours a day and are never late.

If these conditions are met, then the employee receives a bonus. This is an example of a biconditional (bi-implication), where both conditions are mutually connected.

The Source Code

```

5. Lab Exercise 5 Logika V- MatKom.py > axioms_theorems_demo
1  def opbiconditional(work_hours, is_late):
2      # Karyawan mendapat bonus jika bekerja lebih dari 8 jam dan tidak terlambat
3      return (work_hours > 8) == (not is_late)
4
5  def biconditional_demo():
6      # Studi kasus
7      work_hours = 9 # Jam kerja lebih dari 8
8      is_late = False # Tidak terlambat
9
10     # Penerapan
11     result = opbiconditional(work_hours, is_late)
12     if result:
13         print("Karyawan layak mendapatkan bonus.")
14     else:
15         print("Karyawan tidak layak mendapatkan bonus.")
16     print("=====\n")
17

```

The function returns the biconditional $(\text{work_hours} > 8) == (\text{not is_late})$, true only when both sides have the same truth value. The function also sets $\text{work_hours}=9$, $\text{is_late}=\text{False}$, calls `opbiconditional`, and prints an eligibility message.

The Output

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter
Lab Excercise 5 Logika V- MatKom.py"
Karyawan layak mendapatkan bonus.
=====
```

opbiconditional is True, “Karyawan layak mendapatkan bonus.” then the separator line.

2. Case Study Inference

If it rains, then the ground will be wet. When we see that the ground is wet, we can conclude that it has rained. This is an example of inference.

The Source Code

```
5. Lab Excercise 5 Logika V- MatKom.py > axioms_theorems_demo
18 def opinferensi(is_raining):
19     # Jika hujan, maka tanah basah
20     if is_raining:
21         return "Tanah basah"
22     else:
23         return "Tanah kering"
24
25 def inference_demo():
26     # Studi kasus
27     is_raining = True # Hujan turun
28
29     # Penerapan
30     result = opinferensi(is_raining)
31     print(f"Hasil inferensi: {result}")
32     print("=====\n")
33
```

The function sets is_raining=True, calls opinferensi, prints the returned text.

The Output

```
Hasil inferensi: Tanah basah
=====
```

For is_raining=True (from inference_demo), it returns "Tanah basah". It prints: “Hasil inferensi: Tanah basah” and the separator.

3. Case Study Argument

When given two argument statements as follows:

- If the exam score is greater than 70, then you pass.
- You have a score of 85.

With these two arguments, we can conclude that you pass.

The Source Code

```
34 def opargumen(nilai):
35     # Jika nilai lebih dari 70, maka lulus
36     if nilai > 70:
37         return "Lulus"
38     else:
39         return "Tidak lulus"
40
41 def argument_demo():
42     # Studi kasus
43     nilai = 85 # Nilai lebih dari 70
44
45     # Penerapan
46     result = opargumen(nilai)
47     print(f"Hasil argumen: {result}")
48     print("=====\n")
```

Implements the grading rule: return "Lulus" if nilai > 70, else "Tidak lulus". Then this demo assigns nilai=85, calls opargumen, and prints the result string.

The Output

```
Hasil argumen: Lulus
=====
```

With nilai=85 (set in argument_demo), it returns "Lulus". Then it prints: "Hasil argumen: Lulus" followed by the separator.

4. Case Study Axioms, Theorems, Lemma, and Corollary

- Axiom: A basic rule accepted without proof, for example: *"Every even number can be divided by 2."*
- Theorem: A statement that can be proven based on axioms, for example: *"The sum of two even numbers is an even number."*
- Lemma: A smaller statement used to prove a theorem, for example: *"An even number is a multiple of 2."*
- Corollary: A direct result of a theorem, for example: *"If two odd numbers are added, the result is an even number."*

The Source Code

```

5. Lab Exercise 5 Logika V- MatKom.py > axioms_theorems_demo
49
50 # Aksioma: Setiap angka genap dapat dibagi 2
51 def opaksioma(n):
52     return n % 2 == 0
53
54 # Lemma: Bilangan genap adalah kelipatan dari 2
55 def oplemma(n):
56     return opaksioma(n)
57
58 # Teorema: Jumlah dua bilangan genap adalah bilangan genap
59 def opteorema(a, b):
60     return opaksioma(a + b)
61
62 # Corollary: Jika dua bilangan ganjil dijumlahkan, hasilnya bilangan genap
63 def opcorollary(a, b):
64     return opaksioma(a + b)
65
66 def axioms_theorems_demo():
67     # Studi kasus
68     a = 4 # Bilangan genap
69     b = 6 # Bilangan genap
70     c = 3 # Bilangan ganjil
71     d = 5 # Bilangan ganjil
72
73     # Penerapan
74     print(f"Aksioma: {a} adalah bilangan genap? {opaksioma(a)}")
75     print(f"Lemma: {b} adalah kelipatan 2? {oplemma(b)}")
76     print(f"Teorema: Jumlah {a} dan {b} adalah bilangan genap? {opteorema(a, b)}")
77     print(f"Corollary: Jumlah {c} dan {d} adalah bilangan genap? {opcorollary(c, d)}")
78     print("=====\n")
79

```

The program defines `opaksioma(n)` to implement the axiom “even numbers are divisible by 2” via `n % 2 == 0`; `oplemma(n)` simply reuses `opaksioma(n)` to express the lemma “multiples of 2 are even”; `opteorema(a, b)` encodes the theorem “the sum of two even numbers is even” by checking `opaksioma(a + b)`; `opcorollary(a, b)` expresses the corollary “the sum of two odd numbers is even,” also by testing `opaksioma(a + b)`; and `axioms_theorems_demo()` chooses `a=4`, `b=6`, `c=3`, `d=5`, calls those helpers, and prints the labeled results.

The Output

```

Aksioma: 4 adalah bilangan genap? True
Lemma: 6 adalah kelipatan 2? True
Teorema: Jumlah 4 dan 6 adalah bilangan genap? True
Corollary: Jumlah 3 dan 5 adalah bilangan genap? True
=====

```

With the demo values, the console reports the axiom as True for 4, the lemma as True for 6, the theorem as True because $4 + 6 = 10$ is even, and the corollary as True because $3 + 5 = 8$ is even, each line printed with its label, followed by the separator.

PRACTICUM Part 11: Practical Work about Biconditional (Bi-implication), Inference, Arguments, Axioms, Theorems, Lemmas, and Corollaries

Modifying values in Biconditional (Bi-implication), Inference, Arguments, Axioms, Theorems, Lemmas, and Corollaries, including:

1. Biconditional logic case function

The Source Code

a. Device A = True & Device B = True

```
4. Lab Exercise 4 Logika IV- MatKom.py > inference_case
1 def biconditional_logic_case():
2     print("=== Studi Kasus Bikondisional (Bi-implikasi) ===")
3     perangkat_A = True # Perangkat A berfungsi
4     perangkat_B = True # Perangkat B berfungsi
5
6     # A <-> B setara dengan (A & B) ∨ (¬A & ¬B)
7     bikondisional = (perangkat_A and perangkat_B) or (not perangkat_A and not perangkat_B)
8
9     print(f"Perangkat A: {perangkat_A}, Perangkat B: {perangkat_B}, A <-> B: {bikondisional}")
10    print("=====\n")
11
```

The function sets perangkat_A=True and perangkat_B=True, then computes the biconditional with bikondisional = (A and B) or (not A and not B), which is the standard form of $A \leftrightarrow B$ (true exactly when A and B share the same truth value), and prints both states and the computed result.

b. Device A = False & Device B = False

```
4. Lab Exercise 4 Logika IV- MatKom.py > biconditional_logic_case
1 def biconditional_logic_case():
2     print("=== Studi Kasus Bikondisional (Bi-implikasi) ===")
3     perangkat_A = False # Perangkat A tidak berfungsi
4     perangkat_B = False # Perangkat B tidak berfungsi
5
6     # A <-> B setara dengan (A & B) ∨ (¬A & ¬B)
7     bikondisional = (perangkat_A and perangkat_B) or (not perangkat_A and not perangkat_B)
8
9     print(f"Perangkat A: {perangkat_A}, Perangkat B: {perangkat_B}, A <-> B: {bikondisional}")
10    print("=====\n")
11
```

This version keeps the same biconditional formula but assigns perangkat_A=False and perangkat_B=False; since both operands match, the expression still evaluates to true, and the code prints the two states together with the biconditional outcome.

The Output

a. Device A = True & Device B = True

```
[Running] python -u "c:\Users\USER\OneDrive\Documents\
Lab Exercise 4 Logika IV- MatKom.py"
=== Studi Kasus Bikondisional (Bi-implikasi) ===
Perangkat A: True, Perangkat B: True, A <-> B: True
=====
```

The console shows “Perangkat A: True, Perangkat B: True, $A \leftrightarrow B$: True,” confirming that when both devices are ON the biconditional holds, meaning the devices are synchronized/consistent with the rule.

b. Device A = False & Device B = False

```
[Running] python -u "c:\Users\USER\OneDrive\Documents\Ku
Lab Excercise 4 Logika IV- MatKom.py"
=== Studi Kasus Bikondisional (Bi-implikasi) ===
Perangkat A: False, Perangkat B: False,  $A \leftrightarrow B$ : True
=====
```

The console shows “Perangkat A: False, Perangkat B: False, $A \leftrightarrow B$: True,” confirming that when both devices are OFF the biconditional also holds; the system remains synchronized (it would be false only if one were True and the other False).

The Analysis

The code models a biconditional for two devices A and B using the identity $A \leftrightarrow B \equiv (A \wedge B) \vee (\neg A \wedge \neg B)$, then prints each device’s state and the computed result. In the first case both are set to True, and the output correctly reports $A \leftrightarrow B$: True; in the second case both are set to False, and the output again reports $A \leftrightarrow B$: True. These two runs demonstrate the rule that the biconditional is satisfied precisely when A and B share the same state—both ON or both OFF—implying the system is synchronized; by contrast (though not shown), any mixed state (True, False or False, True) would make $A \leftrightarrow B$ evaluate to False, indicating lack of synchronization.

2. Inference case function

The Source Code

a. Motion Sensor = True & Alarm = True

```
4. Lab Exercise 4 Logika IV- MatKom.py > argument_case
13 def inference_case():
14     print("=== Studi Kasus Inferensi ===")
15     sensor_gerak = True # Sensor mendeteksi gerakan
16     alarm_berbunyi = True # Alarm berbunyi
17
18     # p -> q setara dengan ~p v q
19     implication = (not sensor_gerak) or alarm_berbunyi
20
21     print(f"Sensor mendeteksi gerakan: {sensor_gerak}, Alarm berbunyi: {alarm_berbunyi}")
22     print(f"p -> q: {implication}")
23     if sensor_gerak and implication:
24         print("Alarm harus berbunyi!")
25     else:
26         print("Sistem gagal mendeteksi pergerakan!")
27     print("=====\n")
28
```

The function sets `sensor_gerak=True` and `alarm_berbunyi=True`, computes the implication $p \rightarrow q$ as `implication = (not sensor_gerak) or alarm_berbunyi`, then prints the states and the truth of the implication; because the sensor detects motion and the alarm rings, the implication is True, so the guard if `sensor_gerak and implication:` is satisfied and the branch that prints “Alarm harus berbunyi!” will execute.

b. Motion Sensor = False & Alarm = True

```
4. Lab Exercise 4 Logika IV- MatKom.py > inference_case
13 def inference_case():
14     print("=== Studi Kasus Inferensi ===")
15     sensor_gerak = False # Sensor tidak mendeteksi gerakan
16     alarm_berbunyi = True # Alarm berbunyi
17
18     # p -> q setara dengan ~p V q
19     implication = (not sensor_gerak) or alarm_berbunyi
20
21     print(f"Sensor mendeteksi gerakan: {sensor_gerak}, Alarm berbunyi: {alarm_berbunyi}")
22     print(f"p -> q: {implication}")
23     if sensor_gerak and implication:
24         print("Alarm harus berbunyi!")
25     else:
26         print("Sistem gagal mendeteksi pergerakan!")
27     print("=====\n")
28
```

Here the sensor does not detect motion (False) while the alarm is True; the implication (not False) or True evaluates to True (implications are vacuously true when the antecedent is false), but the if `sensor_gerak and implication:` guard fails because `sensor_gerak` is False, so the else branch runs and prints the fallback message.

The Output

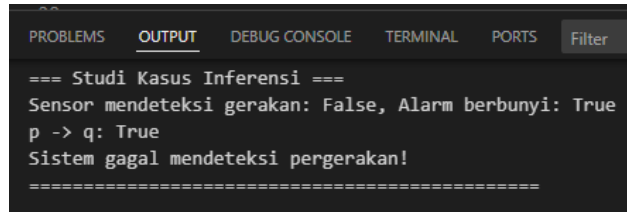
a. Motion Sensor = True & Alarm = True

```
=== Studi Kasus Inferensi ===
Sensor mendeteksi gerakan: True, Alarm berbunyi: True
p -> q: True
Alarm harus berbunyi!
=====
```

The console shows “Sensor mendeteksi gerakan: True, Alarm berbunyi: True” and “p -> q: True,” followed by “Alarm harus berbunyi!”, confirming that when motion is detected and the alarm

rings, the logical rule $p \rightarrow q$ holds and the expected action message is produced.

b. Motion Sensor = False & Alarm = True



```
=== Studi Kasus Inferensi ===
Sensor mendeteksi gerakan: False, Alarm berbunyi: True
p -> q: True
Sistem gagal mendeteksi pergerakan!
=====
```

The console reports “Sensor mendeteksi gerakan: False, Alarm berbunyi: True” and “p -> q: True,” but then prints “Sistem gagal mendeteksi pergerakan!” because the if condition requires the sensor to be True; logically the implication remains true (no motion doesn’t violate the rule), yet the code’s message reflects that no motion was detected so the “alarm must ring” branch is not taken.

The Analysis

The code models the implication “if the sensor detects motion (p) then the alarm rings (q)” using the identity $p \rightarrow q \equiv (\neg p \vee q)$, prints the current states, and then branches on if sensor_gerak and implication: to show an action message. In the first run (p=True, q=True) the implication is True and the guard is satisfied, so the output confirms the rule and prints “Alarm harus berbunyi!”. In the second run (p=False, q=True) the implication is still True (vacuously true because the antecedent is false), but the guard fails since sensor_gerak is False, so the else branch prints “Sistem gagal mendeteksi pergerakan!”. Logically, nothing is wrong with the rule in the second case, the system hasn’t violated $p \rightarrow q$; the message simply reflects that motion wasn’t detected, not that the implication failed (a clearer message would note “no motion detected; rule still satisfied”).

3. Argument case function

Modifying the values into:

- Project Completed on Time = True
- Client Satisfied = True
- Company Receives Bonus = True

The Source Code

```

30 def argument_case():
31     print("=== Studi Kasus Argumen ===")
32     proyek_selesai_tepat_waktu = True # Tim menyelesaikan proyek tepat waktu
33     klien_puas = True # Klien puas
34     perusahaan_terima_bonus = True # Perusahaan menerima bonus
35
36     # Premis 1:  $p \rightarrow q \equiv (\neg p \vee q)$ 
37     premise_1 = (not proyek_selesai_tepat_waktu) or klien_puas
38     # Premis 2:  $q \rightarrow r \equiv (\neg q \vee r)$ 
39     premise_2 = (not klien_puas) or perusahaan_terima_bonus
40     # Kesimpulan:  $p \rightarrow r \equiv (\neg p \vee r)$ 
41     conclusion = (not proyek_selesai_tepat_waktu) or perusahaan_terima_bonus
42
43     print(f"Premis 1 ( $p \rightarrow q$ ): {premise_1}")
44     print(f"Premis 2 ( $q \rightarrow r$ ): {premise_2}")
45     print(f"Kesimpulan ( $p \rightarrow r$ ): {conclusion}")
46     print("=====\n")
47

```

The function models a hypothetical syllogism in a business scenario by setting $p=\text{True}$ (*project finished on time*), $q=\text{True}$ (*client satisfied*), and $r=\text{True}$ (*company gets a bonus*), then computing each implication with the standard equivalence $p \rightarrow q \equiv (\neg p \vee q)$: $\text{premise_1} = (\text{not } p) \text{ or } q$ for $p \rightarrow q$, $\text{premise_2} = (\text{not } q) \text{ or } r$ for $q \rightarrow r$, and $\text{conclusion} = (\text{not } p) \text{ or } r$ for $p \rightarrow r$; finally it prints the truth values of both premises and the inferred conclusion.

The Output

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
=====
=== Studi Kasus Argumen ===
Premis 1 ( $p \rightarrow q$ ): True
Premis 2 ( $q \rightarrow r$ ): True
Kesimpulan ( $p \rightarrow r$ ): True
=====

```

The console shows Premis 1: True, Premis 2: True, and Kesimpulan: True, confirming that with $p=\text{True}$, $q=\text{True}$, $r=\text{True}$ both premises hold and the conclusion $p \rightarrow r$ also holds, which is exactly what the hypothetical syllogism guarantees—when $p \rightarrow q$ and $q \rightarrow r$ are true, $p \rightarrow r$ follows.

The Analysis

The snippet implements a hypothetical syllogism: if the project finishes on time then the client is satisfied ($p \rightarrow q$), and if the client is satisfied then the company gets a bonus ($q \rightarrow r$); therefore, if the project finishes on time then the company gets a bonus ($p \rightarrow r$). Each implication is evaluated via the equivalence $x \rightarrow y \equiv (\neg x \vee y)$, and with the chosen facts $p=\text{True}$, $q=\text{True}$, $r=\text{True}$, both premises compute to True and the conclusion $p \rightarrow r$ also computes to True, matching the console output. This confirms the argument is valid in form (whenever $p \rightarrow q$ and $q \rightarrow r$ are true, $p \rightarrow r$ must be

true) and sound in this instance because the premises are in fact true under the provided assignments.

4. Assignment: Application of Biconditional Theory in Computer Networks

The management of a network consists of two devices (p and q). These devices will only function correctly if both are turned ON at the same time or both are turned OFF.

Implement the concept of Biconditional to test whether the two devices are working in sync.

Create a Python function that receives the status of devices p and q (True or False) and checks whether both are ON/OFF simultaneously.

Display the result and provide an analysis in the following format:

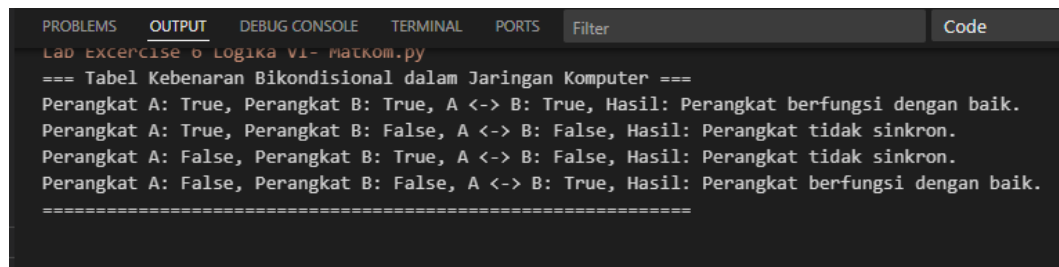
- If both are the same: "Devices are functioning properly."
- If one device is ON and the other is OFF: "Devices are not synchronized."

The Source Code

```
6. Lab Exercise 6 Logika VI- MatKom.py > ...
1  def biconditional_logic_all_cases():
2      print("=== Tabel Kebenaran Bikondisional dalam Jaringan Komputer ===")
3      kasus = [(True, True), (True, False), (False, True), (False, False)]
4
5      for p, q in kasus:
6          bikondisional = (p and q) or (not p and not q)
7          if bikondisional:
8              hasil = "Perangkat berfungsi dengan baik."
9          else:
10             hasil = "Perangkat tidak sinkron."
11
12         print(f"Perangkat A: {p}, Perangkat B: {q}, A <-> B: {bikondisional}, Hasil: {hasil}")
13     print("=====\\n")
14
```

The function `biconditional_logic_all_cases()` prints a header, then iterates over all four boolean pairs for devices A and B: (True, True), (True, False), (False, True), and (False, False). For each pair it computes the biconditional with `bikondisional = (p and q) or (not p and not q)`, which is the standard form of $A \leftrightarrow B$ (true exactly when A and B share the same truth value). It maps that result to a status string—"Perangkat berfungsi dengan baik." if True, otherwise "Perangkat tidak sinkron.", and prints one formatted summary line per case, followed by a separator.

The Output



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter Code
Lab Exercise 6 Logika VI- Matkom.py
=== Tabel Kebenaran Bikondisional dalam Jaringan Komputer ===
Perangkat A: True, Perangkat B: True, A <-> B: True, Hasil: Perangkat berfungsi dengan baik.
Perangkat A: True, Perangkat B: False, A <-> B: False, Hasil: Perangkat tidak sinkron.
Perangkat A: False, Perangkat B: True, A <-> B: False, Hasil: Perangkat tidak sinkron.
Perangkat A: False, Perangkat B: False, A <-> B: True, Hasil: Perangkat berfungsi dengan baik.
=====
```

The console lists the four evaluations in order: A=True, B=True yields $A \leftrightarrow B = \text{True}$ and “Perangkat berfungsi dengan baik.”; A=True, B=False yields $A \leftrightarrow B = \text{False}$ and “Perangkat tidak sinkron.”; A=False, B=True yields $A \leftrightarrow B = \text{False}$ and “Perangkat tidak sinkron.”; and A=False, B=False yields $A \leftrightarrow B = \text{True}$ and “Perangkat berfungsi dengan baik.” This matches the biconditional truth table: only matching states (both ON or both OFF) satisfy the condition.

The Analysis

The code iterates through all four boolean pairs for devices A and B, computes the biconditional with (p and q) or (not p and not q) (i.e., $A \leftrightarrow B$ is true exactly when A and B share the same state), and prints a status message mapped to that truth value; the output confirms the logic: when both devices are True/True or False/False, the biconditional is True and the system is reported as “Perangkat berfungsi dengan baik,” while the mixed states True–False and False–True make the biconditional False and are labeled “Perangkat tidak sinkron,” exactly matching the expected truth table for a biconditional.

5. Assignment: Application of Inference Theory in a Security Alarm System

In a security alarm system, the motion sensor (p) activates the alarm (q). If the sensor detects movement, then the alarm must sound.

Implement Inference to ensure that if the sensor detects movement, then the alarm sounds.

Create a function that receives input on whether the sensor detects movement (True/False) and whether the alarm sounds (True/False).

Use logical inference statements to check if the system works according to the rule.

Display the result and provide an analysis in the following format:

- If inference fails (the sensor detects movement but the alarm does not sound), print "System failure."
- If inference is valid, print "System is functioning properly."

The Source Code

```
6. Lab Exercise 6 Logika VI- MatKom.py > inference_all_cases
14
15 def inference_all_cases():
16     print("=== Tabel Kebenaran Inferensi pada Sistem Alarm ===")
17     kasus = [(True, True), (True, False), (False, True), (False, False)]
18
19     for sensor_gerak, alarm_berbunyi in kasus:
20         # p -> q setara dengan ¬p ∨ q
21         implication = (not sensor_gerak) or alarm_berbunyi
22
23         if implication:
24             hasil = "Sistem berfungsi dengan baik."
25         else:
26             hasil = "Sistem gagal."
27
28     print(f"Sensor: {sensor_gerak}, Alarm: {alarm_berbunyi}, p -> q: {implication}, Hasil: {hasil}")
29     print("-----\n")
30
```

The function iterates over all four (sensor_gerak, alarm_berbunyi) pairs: (True, True), (True, False), (False, True), (False, False). For each pair it evaluates the implication $p \rightarrow q$ with $\text{implication} = (\text{not sensor_gerak}) \text{ or alarm_berbunyi}$ (the standard form $\neg p \vee q$). It maps the boolean to a message "Sistem berfungsi dengan baik." when the implication is True, otherwise "Sistem gagal." and prints one formatted line showing the sensor state, alarm state, the truth of $p \rightarrow q$, and the resulting message, followed by a separator.

The Output

```
[Running] python -u "c:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\Matematika Kom
6 Logika VI- MatKom.py"
=== Tabel Kebenaran Inferensi pada Sistem Alarm ===
Sensor: True, Alarm: True, p -> q: True, Hasil: Sistem berfungsi dengan baik.
Sensor: True, Alarm: False, p -> q: False, Hasil: Sistem gagal.
Sensor: False, Alarm: True, p -> q: True, Hasil: Sistem berfungsi dengan baik.
Sensor: False, Alarm: False, p -> q: True, Hasil: Sistem berfungsi dengan baik.
=====
```

The console lists the four cases exactly as the implication truth table predicts: (True, True) gives $p \rightarrow q = \text{True} \rightarrow$ "Sistem berfungsi dengan

baik.”; (True, False) is the only failing case so $p \rightarrow q = \text{False} \rightarrow$ “Sistem gagal.”; (False, True) and (False, False) both yield $p \rightarrow q = \text{True}$ (vacuous truth when the antecedent is false) and are reported as “Sistem berfungsi dengan baik.”

The Analysis

The results validate the implication “if the sensor detects motion (p) then the alarm rings (q)” using the identity $p \rightarrow q = \neg p \vee q$: across all four (p,q) pairs, the only failure is (True, False), where motion is detected but the alarm does not ring; the other three cases, (True, True), (False, True), and (False, False), all satisfy the rule because either the alarm rings when required or there was no requirement (vacuous truth when $p=\text{False}$). The printed messages correctly reflect this logic: “System works well” whenever the implication is true, and “System failed” only in the single violating case.

6. Assignment: Application of Argument Theory in Business Processes

In a company:

- If the project is completed on time (p), then the client will be satisfied (q).
- If the client is satisfied (q), then the company will receive a bonus (r).

Using these two premises, we can conclude:

If the project is completed on time, then the company will receive a bonus.

Task Implementation

- Create a function that accepts inputs for project status (True/False), client satisfaction (True/False), and company bonus (True/False).
- Check the consistency between the premises and the conclusion.
- Display the results and analysis in the following format:
 - If the conclusion is valid, print: "Conclusion: The company receives a bonus."
 - If the conclusion is invalid, print: "Conclusion: No bonus."

The Source Code

```
6. Lab Exercise 6 Logika VI- MatKom.py > ...
31 def argument_all_cases():
32     print("=== Studi Kasus Argumen dalam Proses Bisnis ===")
33     kasus = [
34         (True, True, True),
35         (True, True, False),
36         (True, False, True),
37         (True, False, False),
38         (False, True, True),
39         (False, True, False),
40         (False, False, True),
41         (False, False, False)
42     ]
43
44     for p, q, r in kasus:
45         # Premis 1: p -> q
46         premise_1 = (not p) or q
47         # Premis 2: q -> r
48         premise_2 = (not q) or r
49         # Kesimpulan: p -> r
50         conclusion = (not p) or r
51
52         # Validasi: jika premis-premis benar, cek apakah kesimpulan juga benar
53         if premise_1 and premise_2 and conclusion:
54             hasil = "Kesimpulan benar: Perusahaan menerima bonus."
55         elif premise_1 and premise_2 and not conclusion:
56             hasil = "Kesimpulan salah: Tidak ada bonus."
57         else:
58             hasil = "Premis tidak terpenuhi, argumen tidak valid."
59
60         print(f"Proyek: {p}, Klien puas: {q}, Bonus: {r} -> {hasil}")
61     print("=====\n")
```

This table enumerates all eight truth assignments for the business argument $p \rightarrow q$, $q \rightarrow r \Rightarrow p \rightarrow r$ (p = project on time, q = client satisfied, r = bonus). Rows marked “Kesimpulan benar: Perusahaan menerima bonus.” are exactly those where both premises hold and the conclusion holds, namely $(p,q,r) = (T,T,T), (F,T,T), (F,F,T), (F,F,F)$. The other rows are labeled “Premis tidak terpenuhi, argumen tidak valid.” because at least one premise fails: (T,T,F) breaks $q \rightarrow r$, $(T,F,*)$ breaks $p \rightarrow q$, and (F,T,F) breaks $q \rightarrow r$. This matches the hypothetical syllogism truth conditions.

The Output

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter Code
6 Logika VI- MatKom.py"
=== Studi Kasus Argumen dalam Proses Bisnis ===
Proyek: True, Klien puas: True, Bonus: True -> Kesimpulan benar: Perusahaan menerima bonus.
Proyek: True, Klien puas: True, Bonus: False -> Premis tidak terpenuhi, argumen tidak valid.
Proyek: True, Klien puas: False, Bonus: True -> Premis tidak terpenuhi, argumen tidak valid.
Proyek: True, Klien puas: False, Bonus: False -> Premis tidak terpenuhi, argumen tidak valid.
Proyek: False, Klien puas: True, Bonus: True -> Kesimpulan benar: Perusahaan menerima bonus.
Proyek: False, Klien puas: True, Bonus: False -> Premis tidak terpenuhi, argumen tidak valid.
Proyek: False, Klien puas: False, Bonus: True -> Kesimpulan benar: Perusahaan menerima bonus.
Proyek: False, Klien puas: False, Bonus: False -> Kesimpulan benar: Perusahaan menerima bonus.
=====
```

This is the same full-case evaluation of the syllogism, reiterating that the argument is valid whenever $p \rightarrow q$ and $q \rightarrow r$ are both true (the four rows above), in which case $p \rightarrow r$ is true as shown; otherwise the program reports the premises are not satisfied and the argument is not valid for that assignment.

The Analysis

The output exhaustively evaluates the argument form $p \rightarrow q, q \rightarrow r \Rightarrow p \rightarrow r$ across all eight truth assignments and shows it is valid: in every row where both premises are true, namely $(p,q,r) = (T,T,T), (F,T,T), (F,F,T), (F,F,F)$, the conclusion $p \rightarrow r$ is also true, and there is no counterexample with true premises and a false conclusion. All failures arise only when a premise is violated: $p \rightarrow q$ fails when $p=T, q=F$, and $q \rightarrow r$ fails when $q=T, r=F$ (e.g., rows $(T,T,F), (T,F,*),$ and (F,T,F)), so the program correctly labels those as “premis tidak terpenuhi/argumen tidak valid.”

7. Assignment: Application of Axioms, Theorems, Lemmas, and Corollaries in Geometry

There are several statements in geometry:

- Axiom: "The sum of the interior angles of a triangle is 180 degrees."
- Theorem: "In a right triangle, one of its angles is 90 degrees."
- Lemma: "The angle opposite the longest side in a right triangle is 90 degrees."
- Corollary: "If a triangle has a 90-degree angle, then it is a right triangle."

Task Implementation

- Create one function that checks whether a triangle is a right triangle based on the values of its three angles.
- If the triangle is a right triangle, print the conclusions derived from the Theorem and Corollary. If not, print "This is not a right triangle."

The Source Code

```

6. Lab Exercise 6 Logika VI- MatKom.py > axioms_theorems_all_cases
63 def axioms_theorems_all_cases():
64     print("=== Studi Kasus Aksioma, Teorema, Lemma, dan Corollary ===")
65     # Beberapa kombinasi contoh segitiga
66     kasus = [
67         (90, 60, 30), # Segitiga siku-siku
68         (60, 60, 60), # Segitiga sama sisi
69         (100, 40, 40), # Segitiga sembarang
70         (120, 30, 30), # Segitiga tumpul
71         (100, 50, 20) # Tidak valid (jumlah ≠ 180)
72     ]
73
74     for s1, s2, s3 in kasus:
75         print(f"\nSudut 1: {s1}, Sudut 2: {s2}, Sudut 3: {s3}")
76
77         # Aksioma: total sudut = 180
78         if s1 + s2 + s3 != 180:
79             print("Kesimpulan: Bukan segitiga (melanggar Aksioma).")
80             continue
81
82         # Teorema & Corollary: ada sudut 90 → segitiga siku-siku
83         if 90 in (s1, s2, s3):
84             print("Kesimpulan (Teorema & Corollary): Ini adalah segitiga siku-siku.")
85         else:
86             print("Kesimpulan: Ini bukan segitiga siku-siku.")
87     print("\n=====")
88

```

The function prepares several angle triples as test cases, and for each set prints the three angles, then applies two checks: first the axiom that a triangle's angles must sum to 180° (if $s1 + s2 + s3 \neq 180 \rightarrow$ print "Bukan segitiga" and continue); second, if the axiom passes, it applies the theorem/corollary that a triangle is right-angled iff one angle is 90° (if 90 in $(s1, s2, s3) \rightarrow$ print "segitiga siku-siku", else "bukan segitiga siku-siku"). A separator is printed at the end.

The Output

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter
=== Studi Kasus Aksioma, Teorema, Lemma, dan Corollary ===

Sudut 1: 90, Sudut 2: 60, Sudut 3: 30
Kesimpulan (Teorema & Corollary): Ini adalah segitiga siku-siku.

Sudut 1: 60, Sudut 2: 60, Sudut 3: 60
Kesimpulan: Ini bukan segitiga siku-siku.

Sudut 1: 100, Sudut 2: 40, Sudut 3: 40
Kesimpulan: Ini bukan segitiga siku-siku.

Sudut 1: 120, Sudut 2: 30, Sudut 3: 30
Kesimpulan: Ini bukan segitiga siku-siku.

Sudut 1: 100, Sudut 2: 50, Sudut 3: 20
Kesimpulan: Bukan segitiga (melanggar Aksioma).

=====

```

The results follow the logic: (90, 60, 30) satisfies the axiom and has a 90° angle, so it's a right triangle; (60, 60, 60) sums to 180° but has no 90° , so not right-angled; (100, 40, 40) and (120, 30, 30) both sum to 180° with no 90° , so not right-angled; and (100, 50, 20) violates the axiom (sum $\neq 180^\circ$), so it's not a triangle.

The Analysis

The run verifies triangle classification by enforcing the axiom ($\sum$ angles = 180°) as a gatekeeper and, only if it holds, applying the theorem/corollary “has a 90° angle $\Rightarrow$ right triangle.” Accordingly, (90,60,30) is right-angled; (60,60,60), (100,40,40), and (120,30,30) are valid but not right-angled; and (100,50,20) violates the axiom so it’s rejected as “not a triangle.” This sequence confirms the logic flow (axiom first prevents misclassification) and shows the theorem/corollary is applied exactly when appropriate.

PRACTICUM Part 12: Application of the Concepts of Definition of Sets, Representation of Sets, Cardinality, Empty Set, Subsets, and Equal Sets

The Source Code for Introduction to Set Theory

```
7. Lab Exercise 7 Logika VII- MatKom.py > ...
1  # Deklarasi himpunan dengan kurung kurawal
2  himpunan_a = {1, 2, 3, 4}
3
4  # Deklarasi himpunan dengan fungsi set()
5  himpunan_b = set([2, 3, 4, 5])
6
7  print(himpunan_a)
8  print(himpunan_b)
9
10 print("=====\n")
```

Declares two sets in Python using curly braces and the set() constructor: himpunan_a = {1,2,3,4} and himpunan_b = set([2,3,4,5]), then demonstrating the two equivalent ways to build a set.

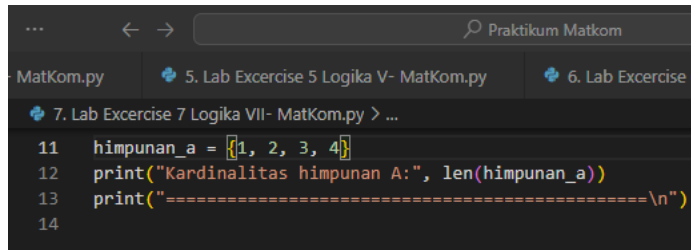
The Output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  Fil
[Running] python -u "c:\Users\USER\OneDrive\Document
7 Logika VII- MatKom.py"
{1, 2, 3, 4}
{2, 3, 4, 5}
=====

[Done] exited with code=0 in 0.818 seconds
```

The console prints the two sets exactly as sets (unordered, unique elements): {1, 2, 3, 4} and {2, 3, 4, 5}, followed by a separator line.

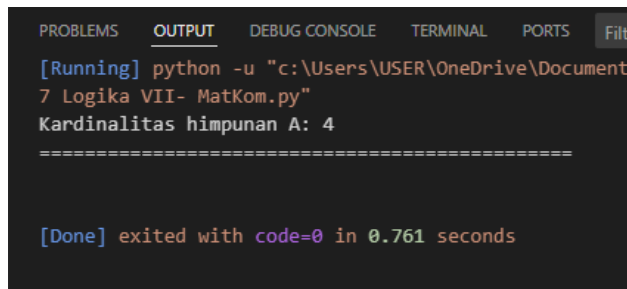
The Source Code for Cardinality of Sets



```
MatKom.py 5. Lab Exercise 5 Logika V- MatKom.py 6. Lab Exercise 6
7. Lab Exercise 7 Logika VII- MatKom.py > ...
11 himpunan_a = {1, 2, 3, 4}
12 print("Kardinalitas himpunan A:", len(himpunan_a))
13 print("=====\n")
14
```

Reuses `himpunan_a = {1,2,3,4}` and prints its cardinality using `len(himpunan_a)`, illustrating how to count the number of distinct elements in a set.

The Output

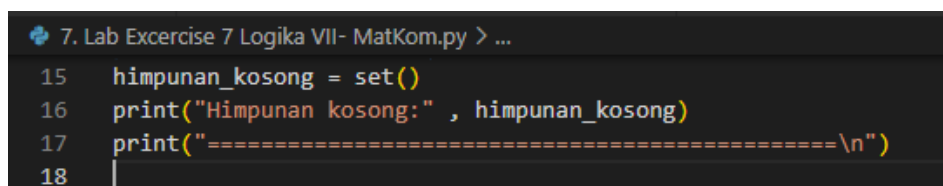


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filt
[Running] python -u "c:\Users\USER\OneDrive\Document
7 Logika VII- MatKom.py"
Kardinalitas himpunan A: 4
=====

[Done] exited with code=0 in 0.761 seconds
```

Shows Kardinalitas himpunan A: 4, confirming that the set {1,2,3,4} contains four unique members.

The Source Code for Empty Set



```
7. Lab Exercise 7 Logika VII- MatKom.py > ...
15 himpunan_kosong = set()
16 print("Himpunan kosong:" , himpunan_kosong)
17 print("=====\n")
18
```

Creates an empty set correctly with `himpunan_kosong = set()` (note that `{}` would make a dict) and prints it to show the literal representation of an empty set.

The Output


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter
[Running] python -u "c:\Users\USER\OneDrive\Documents\Kuliah
7 Logika VII- MatKom.py"
Himpunan kosong: set()
=====

[Done] exited with code=0 in 1.056 seconds
```

Displays Himpunan kosong: set(), verifying the empty-set object and the proper constructor-based creation.

The Source Code for Subsets

```
atKom.py 5. Lab Exercise 5 Logika V- MatKom.py 6. Lab Exercise 6 Logika V
7. Lab Exercise 7 Logika VII- MatKom.py > ...
18
19 himpunan_a = {1, 2, 3}
20 himpunan_b = {1, 2, 3, 4, 5}
21
22 print(himpunan_a.issubset(himpunan_b)) # Output: True
23 print("=====\n")
24
```

Defines `himpunan_a = {1,2,3}` and `himpunan_b = {1,2,3,4,5}`, then checks subset relation with `himpunan_a.issubset(himpunan_b)` to test whether all elements of A are contained in B.

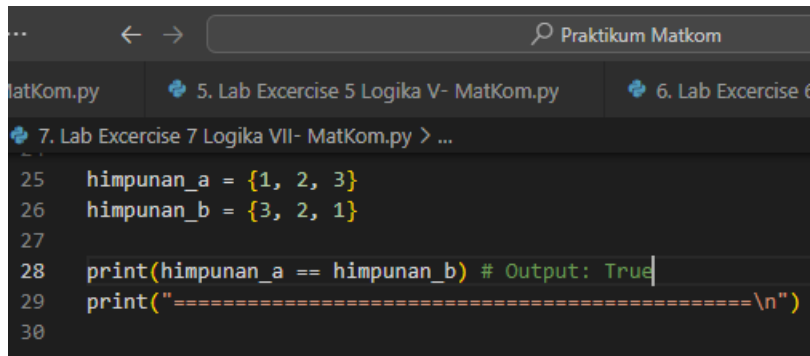
The Output

```
[Running] python -u "c:\Users\USER\OneDrive\Documents\Kuliah
7 Logika VII- MatKom.py"
True
=====

[Done] exited with code=0 in 1.121 seconds
```

Prints True, confirming $\{1,2,3\}$ is indeed a subset of $\{1,2,3,4,5\}$.

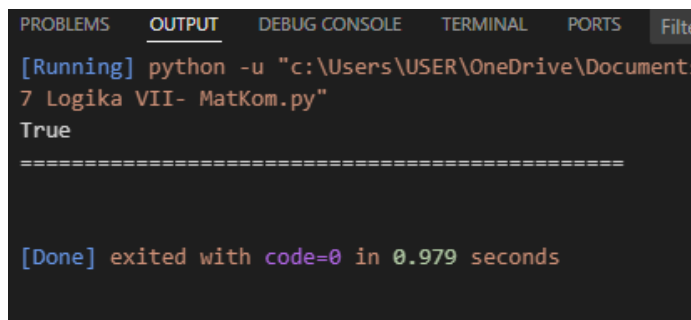
The Source Code for Equal Sets



```
... < -> Praktikum Matkom
MatKom.py 5. Lab Exercise 5 Logika V- MatKom.py 6. Lab Exercise 6
7. Lab Exercise 7 Logika VII- MatKom.py > ...
25 himpunan_a = {1, 2, 3}
26 himpunan_b = {3, 2, 1}
27
28 print(himpunan_a == himpunan_b) # Output: True
29 print("=====\n")
30
```

Sets `himpunan_a = {1,2,3}` and `himpunan_b = {3,2,1}`, then compares equality with `himpunan_a == himpunan_b` to show that order is irrelevant for sets.

The Output



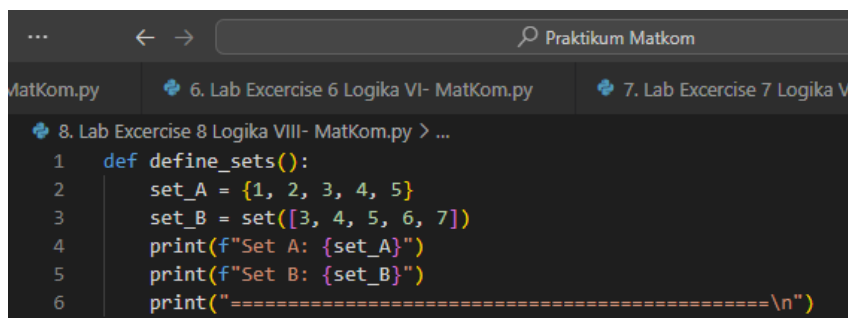
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter
[Running] python -u "c:\Users\USER\OneDrive\Documents\
7 Logika VII- MatKom.py"
True
=====

[Done] exited with code=0 in 0.979 seconds
```

Prints `True`, demonstrating that sets with the same members are equal regardless of insertion or display order.

PRACTICUM Part 13: Application in Python of Definition of Sets, Representation of Sets, Cardinality, Empty Set, Subsets, and Equal Sets

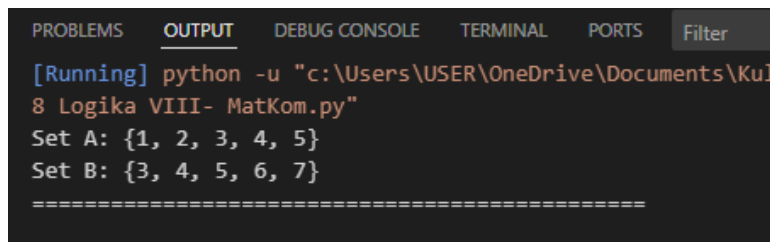
The Source Code for Define Sets



```
... < -> Praktikum Matkom
MatKom.py 6. Lab Exercise 6 Logika VI- MatKom.py 7. Lab Exercise 7 Logika VI
8. Lab Exercise 8 Logika VIII- MatKom.py > ...
1 def define_sets():
2     set_A = {1, 2, 3, 4, 5}
3     set_B = set([3, 4, 5, 6, 7])
4     print(f"Set A: {set_A}")
5     print(f"Set B: {set_B}")
6     print("=====\n")
```

`Define_sets()` creates two sets in two ways—`set_A = {1,2,3,4,5}` via braces and `set_B = set([3,4,5,6,7])` via the constructor—then prints them, illustrating equivalent set declarations and that sets store unique, unordered elements.

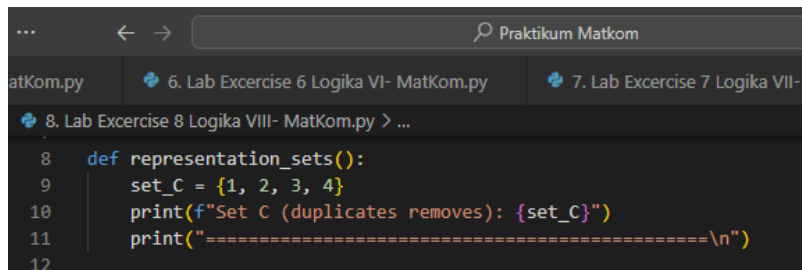
The Output



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter
[Running] python -u "c:\Users\USER\OneDrive\Documents\Ku...
8 Logika VIII- MatKom.py"
Set A: {1, 2, 3, 4, 5}
Set B: {3, 4, 5, 6, 7}
=====
```

The console shows Set A: {1, 2, 3, 4, 5} and Set B: {3, 4, 5, 6, 7}, confirming both creations worked and that set display order is arbitrary while duplicates (if any) would be removed.

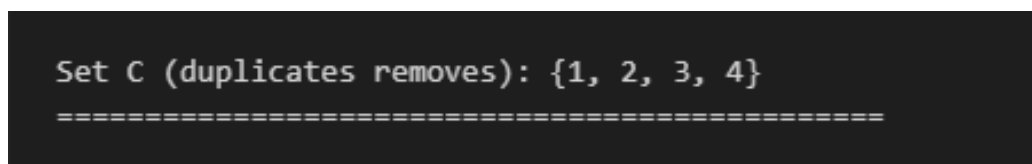
The Source Code for Representation of Sets



```
... < > Praktikum Matkom
atKom.py 6. Lab Exercise 6 Logika VI- MatKom.py 7. Lab Exercise 7 Logika VII- M...
8. Lab Exercise 8 Logika VIII- MatKom.py > ...
8 def representation_sets():
9     set_C = {1, 2, 3, 4}
10    print(f"Set C (duplicates removes): {set_C}")
11    print("=====\\n")
12
```

Defines `set_C = {1,2,3,4}` and prints it to emphasize Python's set representation and the rule that any duplicates in the literal (or input sequence) are automatically discarded.

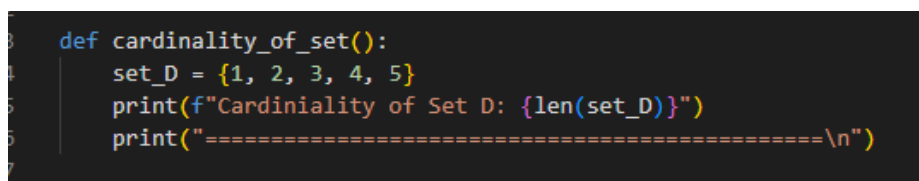
The Output



```
Set C (duplicates removes): {1, 2, 3, 4}
=====
```

It prints Set C (duplicates removes): {1, 2, 3, 4}, demonstrating the canonical, duplicate-free, unordered view of a set.

The Source Code for Cardinality of Set



```
def cardinality_of_set():
    set_D = {1, 2, 3, 4, 5}
    print(f"Cardinality of Set D: {len(set_D)}")
    print("=====\\n")
```

Defines `set_D = {1, 2, 3, 4, 5}` and prints `len(set_D)` to show how to obtain a set's cardinality (the count of distinct members).

The Output

```
Cardinality of Set D: 5
=====
```

The program outputs Cardinality of Set D: 5, confirming there are five unique elements in set_D.

The Source Code for Empty Set

```
18 def empty_set():
19     empty_set = set()
20     print(f"Empty Set: {empty_set}")
21     print("=====\n")
22
```

empty_set() constructs an empty set correctly with empty_set = set() (since {} creates a dict) and prints it, highlighting the proper syntax for the empty set in Python.

The Output

```
Empty Set: set()
=====
```

The console shows Empty Set: set(), verifying an empty set object and its printed form.

The Source Code for Check Subsets

```
22
23 def check_subnets():
24     set_E = {1, 2, 3}
25     set_F = {1, 2, 3, 4, 5}
26     is_subset = set_E.issubset(set_F) # Check E is a subset of F
27     print(f"Is Set E a subset of Set F? {is_subset}")
28     print("=====\n")
29
```

check_subnets() defines set_E = {1,2,3} and set_F = {1,2,3,4,5}, then evaluates set_E.issubset(set_F) to test whether every element of E is contained in F.

The Output

```
Is Set E a subset of Set F? True
=====
```

It prints Is Set E a subset of Set F? True, confirming {1,2,3} is indeed a subset of {1,2,3,4,5}.

The Source Code for Check Equal Sets

```
29
30 def check_equal_sets():
31     set_G = {1, 2, 3}
32     set_H = {3, 2, 1}
33     are_equal = set_G == set_H # Check if sets G and H are equal
34     print(f"Are Set G and Set H equal? {are_equal}")
35     print("=====\n")
36
```

check_equal_sets() defines set_G = {1, 2, 3} and set_H = {3, 2, 1}, then checks equality with set_G == set_H; in Python, set equality ignores order and compares membership only, so this expression returns a boolean indicating whether both sets contain exactly the same elements.

The Output

```
Are Set G and Set H equal? True
=====

[Done] exited with code=0 in 1.68 seconds
```

The program prints Are Set G and Set H equal? True, confirming that {1,2,3} and {3,2,1} are equal as sets because they have the same members regardless of order.

The Source Code for main execution

```
37 if __name__ == "__main__":
38     define_sets()
39     representation_sets()
40     cardinality_of_set()
41     empty_set()
42     check_subnets()
43     check_equal_sets()
```

The if __name__ == "__main__": block acts as the entry point and calls all demo functions in sequence—define_sets, representation_sets, cardinality_of_set, empty_set, check_subnets, and check_equal_sets, so running the script executes each showcase one after another.

PRACTICUM Part 14: Case Study in the Theory of Definition of Sets, Representation of Sets, Cardinality, Empty Set, Subsets, and Equal Sets

1. Case Study Students Present in Both Classes

The use of the intersection operation is to find out which students are present in both classes, namely Computational Mathematics and Data Structures.

The Source Code

```
MatKom.py 7. Lab Exercise 7 Logika VII- MatKom.py 8. Lab Exercise 8 Logika VIII- MatKom.py
9. Lab Exercise 9 Logika IX- MatKom.py > ...
1 matkom = {"Deny", "Budi", "Citra", "Dewi", "Eko"}
2 strukdat = {"Budi", "Citra", "Eko", "Farhan", "Gita"}
3
4 mahasiswa_kedua_kelas = matkom.intersection(strukdat)
5 print(f"Mahasiswa yang mengambil kedua kelas: {mahasiswa_kedua_kelas}")
```

Computes students who take the two classes then prints the resulting set.

The Output

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter
[Running] python -u "c:\Users\USER\OneDrive\Documents\Kuliah\Sema
9 Logika IX- MatKom.py"
Mahasiswa yang mengambil kedua kelas: {'Citra', 'Budi', 'Eko'}

[Done] exited with code=0 in 0.887 seconds
```

Shows the symmetric result { 'Citra', 'Budi', 'Eko' }, i.e., names present in both.

2. Case Study Students Present in Only One Class

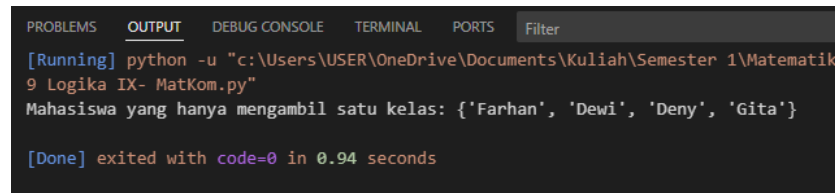
The symmetric_difference operation is used to find students who are present in only one of the two classes, but not in both.

The Source Code

```
6
7 mahasiswa_satu_kelas = matkom.symmetric_difference(strukdat)
8 print(f"Mahasiswa yang hanya mengambil satu kelas: {mahasiswa_satu_kelas}")
```

Computes students who take exactly one of the two classes using the symmetric difference: `mahasiswa_satu_kelas = matkom.symmetric_difference(strukdat)`, then prints the resulting set.

The Output



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter
[Running] python -u "c:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\Matematik
9 Logika IX- MatKom.py"
Mahasiswa yang hanya mengambil satu kelas: {'Farhan', 'Dewi', 'Deny', 'Gita'}

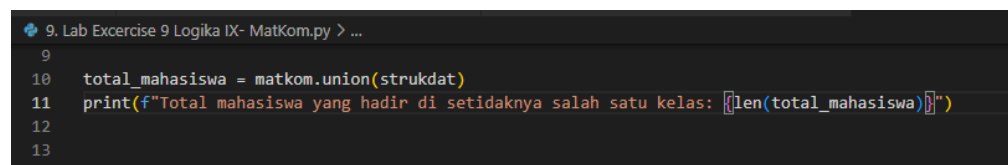
[Done] exited with code=0 in 0.94 seconds
```

Shows the symmetric-difference result { 'Farhan', 'Dewi', 'Deny', 'Gita' }, i.e., names present in only one class and not in both.

3. Case Study Total Students Present in at Least One Class

The union operation is used to combine the attendance from both classes, so that the total number of students present in at least one class can be determined.

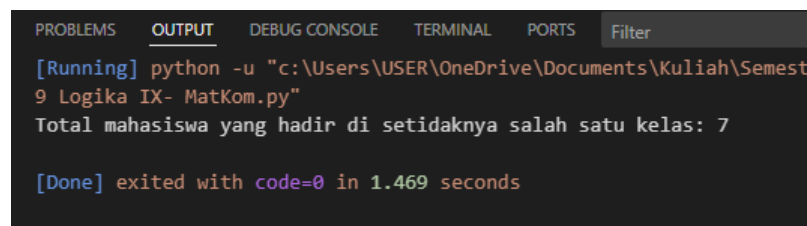
The Source Code



```
9. Lab Exercise 9 Logika IX- MatKom.py > ...
9
10 total_mahasiswa = matkom.union(strukdat)
11 print(f"Total mahasiswa yang hadir di setidaknya salah satu kelas: {len(total_mahasiswa)}")
12
13
```

Builds the union of both classes with `total_mahasiswa = matkom.union(strukdat)` and prints its cardinality using `len(total_mahasiswa)` to count unique students across the two.

The Output



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter
[Running] python -u "c:\Users\USER\OneDrive\Documents\Kuliah\Semest
9 Logika IX- MatKom.py"
Total mahasiswa yang hadir di setidaknya salah satu kelas: 7

[Done] exited with code=0 in 1.469 seconds
```

Displays 7, confirming there are seven distinct students when the rosters are combined without duplicates.

4. Case Study Check Whether All Computational Mathematics Students Attend Data Structures

The subset check function is used to verify whether all students from the Computational Mathematics class are also present in the Data Structures class.

If yes, the result is True; if not, the result is False.

The Source Code

```
9. Lab Exercise 9 Logika IX- MatKom.py > ...
13 matkom_subset_strukdat = matkom.issubset(strukdat)
14 if matkom_subset_strukdat:
15     print("Semua mahasiswa yang hadir di Matematika Komputasi juga hadir di Struktur Data.")
16 else:
17     print("Tidak semua mahasiswa yang hadir di Matematika Komputasi hadir di Struktur Data.")
```

Checks whether Matkom is a subset of Strukdat via `matkom.issubset(strukdat)` and prints a message depending on the boolean result.

The Output

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter Code
[Running] python -u "c:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\Matematika Komputasi\week 04\Prak
9 Logika IX- MatKom.py"
Tidak semua mahasiswa yang hadir di Matematika Komputasi hadir di Struktur Data.

[Done] exited with code=0 in 1.292 seconds
```

Prints “Tidak semua mahasiswa yang hadir di Matematika Komputasi hadir di Struktur Data.” because the subset test is False (Matkom has members not in Strukdat).

5. Case Study Check Whether Attendance in Both Classes is the Same

By comparing the number of students present in both classes using, it can be verified whether the number of students attending the two classes is the same or not. The result of this comparison will indicate whether both groups have the same size.

The Source Code

```
19 if len(matkom) == len(strukdat):
20     print("Jumlah mahasiswa yang hadir di kedua kelas sama.")
21 else:
22     print("Jumlah mahasiswa yang hadir di kedua kelas berbeda.")
```

Compares the sizes of the two classes with `len(matkom) == len(strukdat)` and prints whether the counts are the same or different.

The Output


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter
[Running] python -u "c:\Users\USER\OneDrive\Documents\Ku
9 Logika IX- MatKom.py"
Jumlah mahasiswa yang hadir di kedua kelas sama.

[Done] exited with code=0 in 0.952 seconds
```

Reports “Jumlah mahasiswa yang hadir di kedua kelas sama.” indicating both sets have the same cardinality, even though their members differ.

PRACTICUM Part 15: Practical Work about Definition of Sets, Representation of Sets, Cardinality, Empty Set, Subsets, and Equal Sets

1. Create a Python program that accepts input of student names for two classes (such as Computational Mathematics and Data Structures). Then, use set operations to answer the following questions:
 - Which students are present in both classes?
 - Which students are present in only one of the classes?
 - How many total students are present in at least one of the classes?

The Source Code

```
10. Lab Exercise 10 Logika X- MatKom.py > ...
1  matkom  = {"Deny", "Budi", "Citra", "Dewi", "Eko"}
2  strukdat = {"Budi", "Citra", "Eko", "Farhan", "Gita"}
3
4  # ----- Helper -----
5  def show_section(title: str):
6      print("\n" + "*" * len(title))
7      print(title)
8      print("=" * len(title))
9
10 # ----- Langkah 1: Analisis dua kelas (Matkom vs Strukdat) -----
11 show_section("Langkah 1: Matematika Komputasi vs Struktur Data")
12
13 kedua = matkom.intersection(strukdat)
14 satu_saja = matkom.symmetric_difference(strukdat)
15 total = matkom.union(strukdat)
16
17 print(f"Mahasiswa yang hadir di kedua kelas: {sorted(kedua)}")
18 print(f"Mahasiswa yang hanya hadir di salah satu kelas: {sorted(satu_saja)}")
19 print(f>Total mahasiswa unik (setidaknya salah satu kelas): {len(total)} -> {sorted(total)}")
20
```

The snippet defines two class sets `matkom = {'Deny','Budi','Citra','Dewi','Eko'}` and `strukdat = {'Budi','Citra','Eko','Farhan','Gita'}`, and a `show_section` helper to print a header. It then computes three set operations: `kedua = matkom.intersection(strukdat)` for students in both classes, `satu_saja = matkom.symmetric_difference(strukdat)` for students in exactly one class,

and `total = matkom.union(strukdat)` for all unique students across the two. Each result is printed, with `sorted(...)` for stable ordering and `len(total)` to show the union's size.

The Output

```
[Running] python -u "c:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\Matematika Komputasi\week 04\Praktikum Matkom\
Exercise 10 Logika X- MatKom.py"

=====
Langkah 1: Matematika Komputasi vs Struktur Data
=====
Mahasiswa yang hadir di kedua kelas: ['Budi', 'Citra', 'Eko']
Mahasiswa yang hanya hadir di salah satu kelas: ['Deny', 'Dewi', 'Farhan', 'Gita']
Total mahasiswa unik (setidaknya salah satu kelas): 7 -> ['Budi', 'Citra', 'Deny', 'Dewi', 'Eko', 'Farhan', 'Gita']
```

The console confirms the math: the intersection is ['Budi', 'Citra', 'Eko'] (present in both classes); the symmetric difference is ['Deny', 'Dewi', 'Farhan', 'Gita'] (present in exactly one class); and the union contains 7 unique students ['Budi', 'Citra', 'Deny', 'Dewi',

'Eko', 'Farhan', 'Gita'], meaning there are seven distinct names across the two rosters.

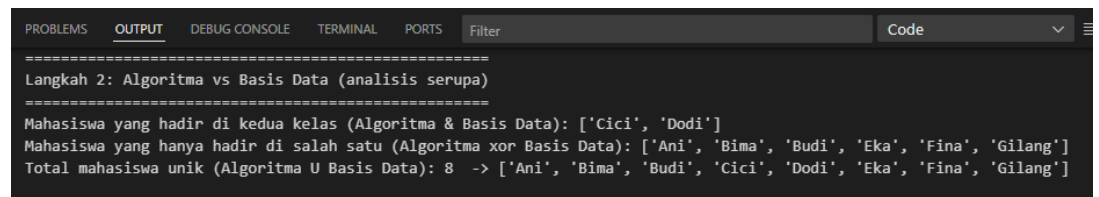
2. Modify the program that has been created to add two additional courses (Algorithms and Databases) with different student name inputs. Perform a similar analysis for these two courses and display the results.

The Source Code

```
10. Lab Exercise 10 Logika X- MatKom.py > ...
21 # ----- Langkah 2: Modifikasi - dua mata kuliah tambahan -----
22
23 # Langkah 2 (dua kelas tambahan, gunakan daftar nama berbeda)
24 algoritma = {"Ani", "Budi", "Cici", "Dodi", "Eka"}
25 basisdata = {"Bima", "Cici", "Dodi", "Fina", "Gilang"}
26
27 show_section("Langkah 2: Algoritma vs Basis Data (analisis serupa)")
28
29 kedua_AB = algoritma.intersection(basisdata)
30 satu_saja_AB = algoritma.symmetric_difference(basisdata)
31 total_AB = algoritma.union(basisdata)
32
33 print(f"Mahasiswa yang hadir di kedua kelas (Algoritma & Basis Data): {sorted(kedua_AB)}")
34 print(f"Mahasiswa yang hanya hadir di salah satu (Algoritma xor Basis Data): {sorted(satu_saja_AB)}")
35 print(f"Total mahasiswa unik (Algoritma U Basis Data): {len(total_AB)} -> {sorted(total_AB)}")
36
```

This block defines two additional class rosters, `algoritma = {'Ani', 'Budi', 'Cici', 'Dodi', 'Eka'}` and `basisdata = {'Bima', 'Cici', 'Dodi', 'Fina', 'Gilang'}`, then prints a section header and computes three standard set operations: `kedua_AB = algoritma.intersection(basisdata)` finds students in both classes, `satu_saja_AB = algoritma.symmetric_difference(basisdata)` finds students in exactly one class, and `total_AB = algoritma.union(basisdata)` gathers all unique students across the two; each result is printed (sorted) and, for the union, the total count is shown.

The Output



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter Code
=====
Langkah 2: Algoritma vs Basis Data (analisis serupa)
=====
Mahasiswa yang hadir di kedua kelas (Algoritma & Basis Data): ['Cici', 'Dodi']
Mahasiswa yang hanya hadir di salah satu (Algoritma xor Basis Data): ['Ani', 'Bima', 'Budi', 'Eka', 'Fina', 'Gilang']
Total mahasiswa unik (Algoritma U Basis Data): 8 -> ['Ani', 'Bima', 'Budi', 'Cici', 'Dodi', 'Eka', 'Fina', 'Gilang']
```

The console reports the intersection as ['Cici', 'Dodi'] (present in both classes), the symmetric difference as ['Ani', 'Bima', 'Budi', 'Eka', 'Fina', 'Gilang'] (present in exactly one class), and the union size 8 with the combined unique list ['Ani', 'Bima', 'Budi', 'Cici', 'Dodi', 'Eka', 'Fina', 'Gilang'], which is consistent with the set calculations.

3. Create a Python program that extends the analysis of student attendance in three classes: "Computational Mathematics", "Data Structures", and "Algorithms".
 - Display the students who are present in all classes using the intersection operation.
 - Display the students who are present in only one class using the symmetric difference operation.
 - Display the total number of students who are present in at least one class using the union operation.
(For points 1–3, use the Data Structures and Algorithms classes).
 - Check whether all students who attend the "Computational Mathematics" class also attend the "Data Structures" class using the subset check function.
 - Check whether the attendance groups of these classes have the same number of students (Equal Sets).

(For points 4–5, use the Computational Mathematics and Data Structures classes).

The Source Code

```

10. Lab Exercise 10 Logika X- MatKom.py > ...
37 # ----- Langkah 3: Analisis tiga kelas -----
38 show_section("Langkah 3: Analisis tiga kelas (Struktur Data & Algoritma)")
39
40 irisan_SD_ALGO = strukdat.intersection(algoritma)
41 print(f"Poin 1 - Hadir di Strukdat dan Algoritma: {sorted(irisn_SD_ALGO)}")
42
43 symdiff_SD_ALGO = strukdat.symmetric_difference(algoritma)
44 print(f"Poin 2 - Hanya di salah satu (Strukdat atau Algoritma): {sorted(symdiff_SD_ALGO)}")
45
46 union_SD_ALGO = strukdat.union(algoritma)
47 print(f"Poin 3 - Total unik (Strukdat U Algoritma): {len(union_SD_ALGO)} -> {sorted(union_SD_ALGO)}")
48
49 show_section("Poin 4 - Subset Check (Matkom dengan Strukdat)")
50 if matkom.issubset(strukdat):
51     print("Semua mahasiswa Matematika Komputasi juga hadir di Struktur Data. (True)")
52 else:
53     print("Tidak semua mahasiswa Matematika Komputasi hadir di Struktur Data. (False)")
54
55 show_section("Poin 5 - Equal Sets (Cardinality)")
56 print(f"|Matkom| = {len(matkom)}, |Strukdat| = {len(strukdat)}")
57 if len(matkom) == len(strukdat):
58     print("Jumlah mahasiswa kedua kelas SAMA.")
59 else:
60     print("Jumlah mahasiswa kedua kelas BERBEDA.")

```

The program finds students in both classes with `strukdat.intersection(algoritma)`, those in exactly one class with `symmetric_difference`, and the unique total with union (printing its count and sorted members). Next, it checks whether $\text{Matkom} \subseteq \text{Strukdat}$ using `issubset` and reports True/False, and finally compares cardinalities via `len(matkom)` vs `len(strukdat)` to say whether the class sizes are the same, note this last check compares sizes only, not full set equality.

The Output

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  Filter  Code
=====
Langkah 3: Analisis tiga kelas (Struktur Data & Algoritma)
=====
Poin 1 - Hadir di Strukdat dan Algoritma: ['Budi']
Poin 2 - Hanya di salah satu (Strukdat atau Algoritma): ['Ani', 'Cici', 'Citra', 'Dodi', 'Eka', 'Eko', 'Farhan', 'Gita']
Poin 3 - Total unik (Strukdat U Algoritma): 9 -> ['Ani', 'Budi', 'Cici', 'Citra', 'Dodi', 'Eka', 'Eko', 'Farhan', 'Gita']

=====
Poin 4 - Subset Check (Matkom dengan Strukdat)
=====
Tidak semua mahasiswa Matematika Komputasi hadir di Struktur Data. (False)

=====
Poin 5 - Equal Sets (Cardinality)
=====
|Matkom| = 5, |Strukdat| = 5
Jumlah mahasiswa kedua kelas SAMA.

```

The output shows that only Budi attends both classes (intersection); the students who attend exactly one class (symmetric difference) are Ani, Cici, Citra, Dodi, Eka, Eko, Farhan, and Gita; the union contains 9 unique students. Ani, Budi, Cici, Citra, Dodi, Eka, Eko, Farhan, Gita; the subset check $\text{Matkom} \subseteq \text{Strukdat}$ is False (some Matkom students are not in Strukdat); and although the members differ, the cardinalities are equal ($|\text{Matkom}| = |\text{Strukdat}| = 5$), so each class has the same number of students.